



Fakultät Informatik

**Entwicklung eines VHDL-basierten Applikationsbeispiels
zur Ablösung des FM350-2 Moduls durch das TM FAST
Modul**

Bachelorarbeit im Studiengang Informatik

vorgelegt von

Jan-Eric Gedicke

Matrikelnummer 3643446

Erstgutachter: Prof. Dr. Meitner

Betreuer: Conrad, Maximilian

© 2025

Inhaltsverzeichnis

1 Einleitung

1.1 Problemstellung und Motivation

Die Produktpalette der Firma Siemens im Bereich der Industrieautomatisierung hat eine auslaufende Produktserie vom Typ SIMATIC S7-300, insbesondere das FM350-2 Modul, sind nicht mehr in der aktiven Vermarktung und dadurch nicht mehr bestellbar. Das FM350-2 Modul bietet 8 Kanäle für hochkanalige Dosieranwendungen und ist nur noch als Ersatzteil verfügbar. Ein alternatives Lösungskonzept ist der Einsatz des TM FAST Moduls, eines freiprogrammierbaren Hochgeschwindigkeits-Moduls, welches eine erweiterte Kanalanzahl von 10 bis 12 Kanälen ermöglicht für die neuere Produktserie vom Typ S7-1500. Ziel dieser Bachelorarbeit ist die Entwicklung eines VHDL-basierten Applikationsbeispiels zur Demonstration der Einsatzmöglichkeiten des TM FAST Moduls in hochkanaligen Dosieranwendungen. Gleichzeitig soll die Integration in das TIA Portal erfolgen, um eine reibungslose Nutzung in industriellen Automatisierungsumgebungen zu gewährleisten.

Als dualer Student bei Siemens verfüge ich bereits über Erfahrung in der industriellen Automatisierungstechnik. Diese Arbeit bietet mir die Gelegenheit, mein Wissen in den Bereichen Embedded Systems, Industrieautomation und Software Engineering weiter zu vertiefen. Ich bin motiviert, dieses praxisnahe Thema in Kooperation mit Siemens zu bearbeiten und einen wertvollen Beitrag zur Weiterentwicklung industrieller Automatisierungslösungen zu leisten.

1.2 Zielsetzung der Arbeit

Ziel dieser Bachelorarbeit ist die Entwicklung eines VHDL-basierten Applikationsbeispiels zur Ablösung des FM350-2 Moduls der alten SIMATIC S7-300-Produktserie durch das TM FAST Modul der neueren S7-1500-Produktserie. Dabei soll die Kanalanzahl von 8 auf die höchst mögliche Anzahl erhöht werden,

um eine flexiblere Nutzung in hochkanaligen Dosieranwendungen zu ermöglichen. Ein wesentlicher Bestandteil der Arbeit ist zudem die Integration des entwickelten Applikationsbeispiels in das TIA Portal, die Programmiersoftware für die industriellen Steuerungen der Firma Siemens, um eine nahtlose Einbindung in bestehende Automatisierungslösungen zu gewährleisten. Neben der Implementierung des VHDL-Codes werden bewährte Methoden des Software Engineerings angewendet, um die Codequalität und Skalierbarkeit sicherzustellen. Schließlich soll eine umfassende Dokumentation erstellt werden, die eine detaillierte Beschreibung der Nutzung und Inbetriebnahme des Moduls umfasst.

1.3 Methodisches Vorgehen und Struktur der Arbeit

Zur Umsetzung der Ziele werden zunächst die bestehenden Funktionalitäten des FM350-2 Moduls analysiert und mit den aktuellen Anforderungen verglichen. Da das TM FAST Modul bereits verfügbar ist, liegt der Schwerpunkt nicht allein auf einer Nachbildung, sondern auf der Nutzung seiner erweiterten Möglichkeiten. Insbesondere soll die höhere Kanaldichte sowie die zusätzliche Funktionalität und Flexibilität des TM FAST berücksichtigt werden.

Im nächsten Schritt erfolgt die Entwicklung des VHDL-basierten Applikationsbeispiels, wobei zentrale Funktionen des FM350-2 Moduls in VHDL umgesetzt und die Kanalanzahl auf so viele wie möglich erweitert wird. Anschließend wird die Integration in das TIA Portal vorgenommen, indem das TM FAST Modul an das Automatisierungssystem angebunden und geeignete Schnittstellen sowie Datenstrukturen entwickelt werden.

Qualitätssicherungsmaßnahmen spielen ebenfalls eine zentrale Rolle. Hierbei werden bewährte Entwicklungswerkzeuge wie Intel Quartus Prime und das TIA Portal genutzt, Teststrategien zur Funktionssicherung implementiert und eine modulare Softwarearchitektur entworfen. Abschließend wird die Dokumentation erstellt, die eine umfassende Beschreibung der Implementierung und Nutzung des Moduls sowie Anwendungsbeispiele und Testberichte enthält. Die Ergebnisse werden in einer Abschlusspräsentation vorgestellt. Im Nachgang wird das entwickelte Applikationsbeispiel Kunden in Form eines Packages zum Download zur Verfügung gestellt.

2 Grundlagen

Um die technischen Aspekte und die Funktionsweise der in dieser Arbeit behandelten Systeme zu verstehen, werden im Folgenden die zugrundeliegenden Konzepte und Technologien erläutert. Relevant sind hierbei die Automatisierungspyramide, SIMATIC S7, Speicherprogrammierbare Steuerungen, Quartus und Questa sowie dem TIA Portal.

2.1 Automatisierungspyramide

Die Automatisierungspyramide stellt ein Modell zur Einordnung von Techniken und Systemen in der industriellen Fertigung dar. Sie umfasst die Kommunikation von der Prozessebene bis zur Unternehmensführung und unterscheidet zwischen horizontalen (innerhalb einer Ebene) und vertikalen (zwischen den Ebenen) Kommunikationsstrukturen. Grundlage ist das OSI-Referenzmodell von 1984, erweitert durch IoT-Konzepte seit 1999, die eine durchgängige Vernetzung von Sensoren, Maschinen und Unternehmenssystemen ermöglichen [?].

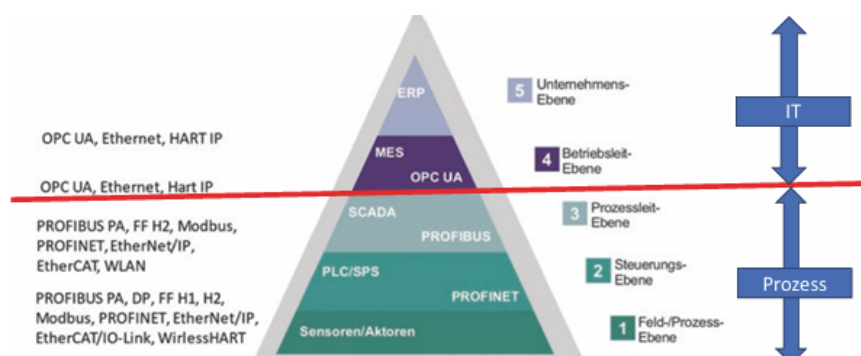


Abbildung 2.1: Automatisierungspyramide, [?]

Ebenen der OT

- **Ebene 1 – Feldebene:** Intelligente Sensoren und Maschinen, die direkt mit der physischen Umgebung interagieren.
- **Ebene 2 – SPS-Ebene:** Speicherprogrammierbare Steuerungen (SPS) und I/O-Einheiten zur Ansteuerung von Aktoren und Verarbeitung der Sensordaten.
- **Ebene 3 – SCADA/HMI-Ebene:** Überwachung und Bedienung über SCADA-Systeme oder Human-Machine-Interfaces (Kontrollraum).

Ebenen der IT

- **Ebene 4 – MES-Ebene:** Manufacturing Execution Systems (MES) für Echtzeit-Produktionssteuerung und Prozessüberwachung.
- **Ebene 5 – ERP-Ebene:** Enterprise Resource Planning (ERP) zur Steuerung der gesamten Wertschöpfungskette „vom Kunden zum Kunden“.

Die Automatisierungspyramide verdeutlicht die Trennung zwischen der OT (Operational Technology: Ebenen 1–3) und der IT (Information Technology: Ebenen 4–5). Sämtliche Ebenen sind im Sinne von IoT kommunikationsfähig und nutzen standardisierte Protokolle wie PROFINET, EtherCAT, Modbus TCP, OPC UA oder Ethernet APL. Nicht-Ethernet-basierte Feldbusse wie PROFIBUS oder HART verlieren seit der Einführung von Ethernet APL zunehmend an Bedeutung.

2.2 Überblick über SIMATIC S7

Speicherprogrammierbare Steuerungen (SPS), im Englischen *Programmable Logic Controllers* (PLC), wie die SIMATIC S7-Serie von Siemens, übernehmen Steuerungs- und Regelungsaufgaben in Maschinen und Anlagen. Sie können unter anderem Antriebe, Ventile, Pumpen und Sensoren ansteuern und zeichnen sich durch ihre freie Programmierbarkeit sowie ihre universelle Einsetzbarkeit aus [?]. Im Folgenden werden Aufbau, Funktionsweise und zentrale Komponenten eines SIMATIC S7-Systems erläutert.

2.2.1 Aufbau eines SIMATIC S7-Systems

Ein SIMATIC S7-System besitzt einen modularen Aufbau und besteht aus mehreren Baugruppen mit spezifischen Funktionen.

Controller (CPU)

Die CPU bildet die zentrale Verarbeitungseinheit der Steuerung. Ihre Hauptaufgabe besteht in der Ausführung des Anwenderprogramms, das zur Umsetzung spezifischer Steuerungs- und Regelungsaufgaben dient [?]. Sie verarbeitet die Signale der Ein- und Ausgabebaugruppen, führt Berechnungen durch und gibt Steuerbefehle an die angeschlossenen Sensoren und Aktoren weiter. Darüber hinaus verfügen SIMATIC S7-Controller über integrierte Kommunikationsschnittstellen, wie Ethernet/PROFINET oder PROFIBUS, die einen Datenaustausch mit weiteren Steuerungen, Bediengeräten oder übergeordneten Systemen (z. B. SCADA oder MES) ermöglichen [?].

Signal- und Technologiebaugruppen

Signalbaugruppen (Eingangs- und Ausgangsbaugruppen) stellen die Verbindung zur Maschine oder Anlage her. Sie erfassen digitale oder analoge Prozesssignale (z. B. Spannungen, Ströme) und geben diese an die CPU weiter bzw. leiten Signale an das Feld zurück.

Technologiebaugruppen (*Technologiemodule*) erweitern die Steuerung um hardwaregestützte Funktionen. Sie verfügen über eigene Prozessoren oder FPGAs und können dadurch zeitkritische Aufgaben (z. B. Hochgeschwindigkeitszählen, präzise Zeitstempel, Frequenz- und Pulsmessungen, Positionserfassung oder schnelle Reaktionsausgänge) unabhängig und deterministisch ausführen. Auf diese Weise werden die durch die CPU-Zykluszeit bedingten Latenzen umgangen, die CPU entlastet und die Systemperformance insgesamt gesteigert [?].

Kommunikationsbaugruppen

Kommunikationsmodule erweitern die Schnittstellen- und Protokollfähigkeit einer Steuerung. Sie ermöglichen beispielsweise zusätzliche Feldbus- oder Industrial-Ethernet-Anbindungen und kapseln die entsprechenden Kommunikationsstacks, wodurch die CPU weiter entlastet wird [?].

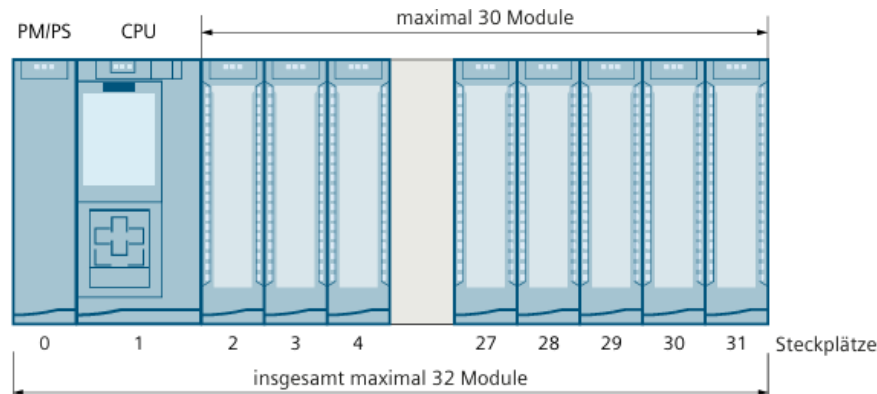


Abbildung 2.2: Aufbau eines SIMATIC S7-Systems [?]

2.2.2 Funktionsweise und Aufbau einer SPS

Eine Speicherprogrammierbare Steuerung (SPS) arbeitet nach dem EVA-Prinzip (Eingabe–Verarbeitung–Ausgabe). Über Eingangsbaugruppen werden die Zustände von Sensoren (z. B. Endschalter, Temperaturfühler, Drehzahlsensoren) erfasst und in einem *Prozessabbild der Eingänge* zwischengespeichert. Dieses Abbild dient als Grundlage für die Bearbeitung durch das Anwenderprogramm. Nach der Verarbeitung werden die Ergebnisse in das *Prozessabbild der Ausgänge* geschrieben und an die Ausgangsbaugruppen übertragen, wodurch die Aktoren (z. B. Motoren, Ventile) angesteuert werden. Dieser Zyklus wiederholt sich kontinuierlich im Millisekundenbereich [?, ?].

Das Anwenderprogramm ist modular aufgebaut und besteht aus *Organisationsbausteinen* (z. B. OB1 als Hauptzyklus), *Funktionsbausteinen* (FB), *Funktionen* (FC) und *Datenbausteinen* (DB). OB1 kann durch höher priorisierte Bausteine wie Interrupt- oder Alarm-OBs unterbrochen werden, wodurch sich die Zykluszeit verlängern kann. Überschreitet die Zykluszeit eine definierte maximale Zykluszeit (Watchdog), geht die SPS in einen Fehlerzustand und beendet die Abarbeitung des Programms (Stopp-Zustand), und alle Ausgänge werden auf 0 gesetzt [?].

Eine modulare SPS besteht mindestens aus einem Baugruppenträger, einer Stromversorgung, einer CPU mit Speicher sowie Ein-/Ausgabebaugruppen. Kompakt-SPS vereinen diese Komponenten in einem Gehäuse. Optional können Kommunikations-, Zähler-, Positionier- oder Regelungsmodule ergänzt werden. Mehrere Baugruppenträger werden über Interface-Module verbunden [?].

Die CPU verfügt über verschiedene Speicherbereiche [?]:

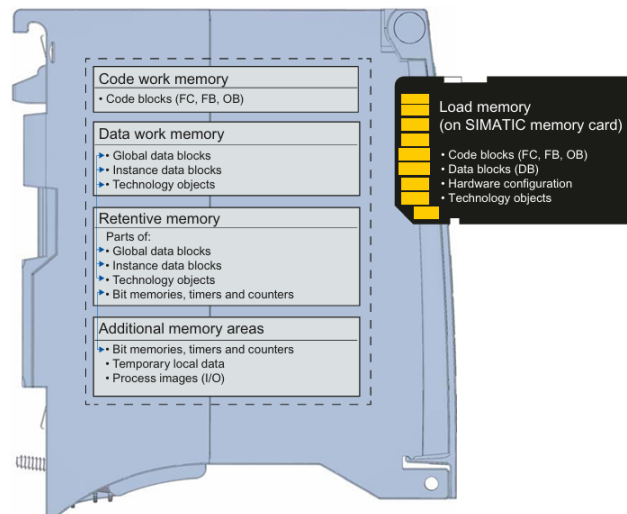


Abbildung 2.3: Speicherbereiche einer SIMATIC S7-CPU [?]

- **Arbeitsspeicher (Anwenderspeicher):** Enthält das aktuell ausgeführte SPS-Programm und die benötigten Datenbausteine.
- **Ladespeicher:** Dient als Zwischenspeicher für Programme und kann über RAM-, EPROM-, FLASH- oder MMC-Karten erweitert werden.
- **Systemspeicher:** Beinhaltet Operandenbereiche wie Eingänge, Ausgänge, Merker, Zeiten und Zähler.
- **Peripherie:** Ermöglicht direkten Zugriff auf reale Ein- und Ausgänge, auch ohne Prozessabbild.

Durch diese Struktur ist die SPS in der Lage, deterministisch und zuverlässig auf geänderte Eingangssignale zu reagieren, Prozesse zu steuern und eine hohe Betriebssicherheit zu gewährleisten [?]. Die Bearbeitung erfolgt dabei im Wesentlichen zyklisch, indem das Anwenderprogramm in festen Zyklen durchlaufen wird. Ergänzend können azyklische Dienste, beispielsweise ereignis- oder zeitgesteuerte Organisationsbausteine (OBs), spezifische Aufgaben außerhalb des regulären Zyklus übernehmen [?, ?].

2.3 Technologiemodule

Die SIMATIC S7-1500 Steuerung ist ein modular aufgebautes System, dessen Funktionalität flexibel skalierbar ist. Zentraler Bestandteil ist die S7-1500 CPU, in der das Anwenderprogramm ausgeführt wird und die Schnittstellen zu weiteren Automatisierungskomponenten bereitstellt. Neben der CPU lässt sich das System durch verschiedene Module erweitern, darunter Interface-, Signal-, Kommunikations- und Technologiebaugruppen. Das Peripheriesystem ET 200MP ermöglicht den Einsatz der S7-1500 IO-Module entweder direkt an der Steuerung oder über einen kompatiblen PROFINET-/PROFIBUS-Teilnehmer wie das Interfacemodul IM 155 PN, das den Datenaustausch zwischen Steuerung und Peripheriemodulen übernimmt. Abbildung ?? zeigt links den zentralen Aufbau mit CPU und vier Signalbaugruppen und rechts einen vergleichbaren dezentralen Aufbau mit Interfacemodul. Die Kommunikation erfolgt hierbei über PROFINET. Die Signalbaugruppen umfassen sowohl digitale als auch analoge Ein- und Ausgänge, die der Steuerung zusätzliche Funktionalität bereitstellen [?].



Abbildung 2.4: Zentraler und dezentraler Aufbau der SIMATIC S7-1500 Steuerung mit ET 200MP Peripheriesystem [?]

Technologiebaugruppen bieten eine hardwarenahe Signalvorverarbeitung für schnelle Zähl- und Messaufgaben sowie präzise Positionserfassung mittels Inkremental- oder SSI-Absolutwertgebern. Bisherige Module wurden über Technologieobjekte oder direkt im Anwenderprogramm im TIA Portal parametriert und programmiert. Das aktuelle Portfolio beinhaltet unter anderem das TM NPU, TM Count 2x24 V, TM Posinput 2, TM Timer DIDQ 16x24 V und das TM PTO 4. Das TM

NPU arbeitet mit einem trainierten neuronalen System und erlaubt den Anschluss verschiedener Sensoren über Gigabit-Ethernet oder USB 3.1. Die Zählerbaugruppe TM Count 2x24 V ist vielseitig einsetzbar für Zähl- und Messaufgaben sowie zur Positionserfassung über Inkrementalgeber. Das TM Posinput 2 Modul stellt zwei Zählkanäle für Differenzeingangssignale nach RS422 mit bis zu 1 MHz bereit und ermöglicht die Positionserfassung mit SSI-Absolutwertgebern. Der TM Timer DIDQ 16x24 V dient zeitkritischen Aufgaben, wie z. B. der präzisen Einhaltung von Reaktionszeiten. Das PTO 4 Modul unterstützt die Ansteuerung von Schrittmotoren und Servoantrieben mit PTI-Schnittstelle [?].

Das TM FAST Modul ist eine FPGA und dadurch freiprogrammierbar. Es ist bereits länger Bestandteil des Portfolios und wird kontinuierlich um neue Funktionen erweitert, die aus konkreten Kundenanwendungsfällen abgeleitet werden, wie zusätzliche Protokoll-, Zähl-, Trigger- und Preprocessing-Bausteine. Diese Funktionen werden als wiederverwendbare Funktionsblöcke bereitgestellt, wodurch bestehende Anforderungen schneller umgesetzt und zukünftige Anwendungsfälle flexibel ergänzt werden können [?].

2.4 Software

Für die Erstellung einer Anwendung auf dem TM FAST werden mehrere Softwareprogramme benötigt. Das TIA Portal übernimmt die Hardwarekonfiguration sowie die Programmierung des Anwenderprogramms, das die CPU steuert und den Funktionsumfang des TM FAST aktiviert. Über das Engineering-Framework kann zudem die Firmware auf das Modul geladen werden. Für die Entwicklung auf dem integrierten Intel® FPGA Cyclone 10 LP wird die Designumgebung Intel® Quartus® Prime Lite eingesetzt. Zur Simulation und Verifikation des Hardwareentwurfs kommt die Questa-Intel® FPGA Starter Edition zum Einsatz.

2.4.1 Totally Integrated Automation Portal

Das *Totally Integrated Automation Portal* (TIA Portal) ist die zentrale Engineering-Plattform für SIMATIC S7-Steuerungen. Es ist das zentrale Tool für die Projektierung, Programmierung, Inbetriebnahme und Diagnose von Automatisierungsprojekten und deckt damit sowohl die Hardwarekonfiguration als auch die Softwa-

reentwicklung ab [?]. Die Umgebung umfasst u. a. SIMATIC STEP 7 für die SPS-Programmierung, SIMATIC WinCC für HMI-Anwendungen, SINAMICS Start-drive zur Antriebskonfiguration sowie die SIMATIC Energy Suite für Energiemonitoring [?, ?].

In dieser Arbeit wird TIA Portal V20 verwendet. Für die Kommunikation zwischen CPU und TM FAST ist eine entsprechende Hardwarekonfiguration erforderlich. Hierbei werden die Komponenten anhand ihrer Artikelnummer ausgewählt und im Projekt verknüpft. Abb. ?? zeigt eine Beispielkonfiguration bestehend aus Powermodul (PM), CPU 1516-3 PN/DP und TM FAST. Die Baugruppen sind über den Rückwandbus verbunden, wodurch eine zusätzliche PROFINET- oder PROFIBUS-Verbindung ist nicht notwendig.



Abbildung 2.5: Hardwarekonfiguration im TIA Portal mit TM FAST

Nach der Konfiguration erhält das TM FAST im Projekt eine eigene Ansicht und wird als Teil des gesamten Aufbaus dargestellt. Unter *Online & Diagnostics* können Modulstatus, Diagnoseinformationen und Firmwareupdates verwaltet werden. Über *Commissioning* lassen sich die aktuelle User-Logik und weitere Kommandos wie das Laden der Anwendung aus dem Flash-Speicher oder die Aktivierung der Debug-Schnittstelle aufrufen. Für die azyklische Kommunikation stehen die TM FAST User Data Records zur Verfügung, die manuell oder über Funktionsbausteine beschrieben und gelesen werden können. Die dafür bereitgestellte TIA-Bibliothek LTMFAST enthält unter anderem die Anweisungen LTMFAST ControlREC (Ist für die Steuerung des TM FAST zuständig), UserReadREC (Liest Daten aus dem TM FAST), UserWriteREC (Schreibt Daten in den TM FAST) und AppDownload (Lädt die Anwendung auf den TM FAST) [?].

2.4.2 Intel® Quartus® Prime

Intel® Quartus® Prime ist eine Entwicklungsumgebung für FPGAs. Sie deckt den gesamten Prozess von der Designerfassung über die Synthese und Optimierung bis hin zur Simulation und Programmerstellung ab. Es existieren drei Editionen – Lite, Standard und Pro – die sich im Funktionsumfang und in der Geräteunterstützung unterscheiden. Für das Cyclone 10 LP im TM FAST wird in dieser Arbeit die Version 22.1std.0 Lite Edition eingesetzt [?].

Ein Quartus-Projekt kann in mehrere Revisionen aufgeteilt werden, die jeweils eine eigene VHDL-Architektur und Konfigurationsdateien beinhalten. Diese Struktur erlaubt es, verschiedene Anwendungen unabhängig voneinander zu entwickeln. Die Kompilation umfasst die Schritte Analyse/Synthese, Fitter und Assembler:

- **Analyse und Synthese:** Überprüfung der logischen Konsistenz, Erzeugung von Zustandsautomaten, Flipflops und Logikzellen aus dem VHDL-Code, sowie Optimierung zur Ressourcenschonung.
- **Fitter:** Zuweisung der Logikfunktionen zu den FPGA-Ressourcen unter Berücksichtigung von Timing- und Routing-Anforderungen.
- **Assembler:** Erzeugung der Ausgabedateien wie `.sof`, `.pof` oder `.rbf`, die zur Programmierung des FPGAs genutzt werden [?].

2.4.3 Questa-Intel® FPGA

Zur Simulation von Hardwareentwürfen stehen zwei Versionen zur Verfügung: die kostenpflichtige Questa-Intel® FPGA sowie die Starter Edition. Beide basieren auf der Siemens EDA Questa Core Software und bieten umfangreiche Funktionen zur Simulation, Verifikation und zum Debugging von FPGA-Designs [?].

Die Simulation erfolgt entweder direkt aus Quartus oder über Simulationsskripte in Questa. Hierzu wird eine Testbench erstellt, die Eingabesignale (Stimuli) bereitstellt und die resultierenden Ausgangssignale überprüft. Typischerweise werden dabei Prozesse definiert, die z. B. ein Taktsignal generieren oder Signalwechsel nachbilden. Abb. ?? zeigt eine beispielhafte Simulation zur Validierung einer Signum-Funktion [?].

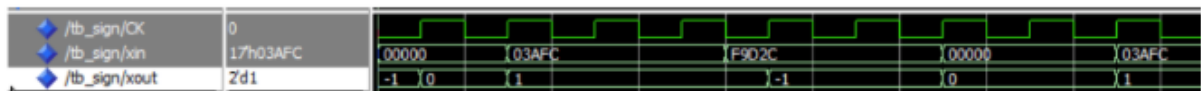


Abbildung 2.6: Zeitdiagramm zur Simulation einer Signum-Funktion in der Questa-Intel® FPGA Starter Edition

2.5 Programmierung einer PLC mit TIA-Portal

Die Grundlage für das *Totally Integrated Automation Portal* (TIA Portal) bildet die Norm IEC 61131-3, die einen einheitlichen Standard für die Softwareentwicklung in speicherprogrammierbaren Steuerungen (SPS) definiert. Diese Norm legt die Struktur und die fünf standardisierten Programmiersprachen fest, wodurch eine herstellerübergreifende Programmierbarkeit ermöglicht wird [?].

Die fünf in IEC 61131-3 beschriebenen Sprachen sind: *Kontaktplan* (KOP), eine grafische Darstellung, die sich an Schaltplänen orientiert; *Funktionsplan* (FUP), eine grafische Sprache zur logischen Verknüpfung von Bausteinen; *Anweisungsliste* (AWL), eine textbasierte Sprache ähnlich zu Assembler; *Strukturierter Text* (ST), eine hochsprachähnliche Programmiersprache vergleichbar mit Pascal; sowie die *Ablaufsprache* (AS) zur Modellierung von Ablaufsteuerungen [?].

Funktionsplan (FUP)

FUP ist eine grafische Programmiersprache, in der logische Verknüpfungen und Operationen durch Symbole dargestellt und miteinander verbunden werden. Typische Anwendungen sind logische Grundfunktionen wie UND-, ODER- oder NICHT-Verknüpfungen. Darüber hinaus können auch komplexere mathematische Operationen oder steuerungsspezifische Aufgaben umgesetzt werden. FUP eignet sich besonders, um Programme übersichtlich und intuitiv zu strukturieren [?].

Strukturierter Text (ST)

SCL ist eine textbasierte Sprache, die in der IEC 61131-3 als *Strukturierter Text* bezeichnet wird. Sie ähnelt der Hochsprache Pascal und ist besonders für komplexe Algorithmen, Berechnungen und strukturierte Programmabläufe geeignet. Mit Schleifen, Verzweigungen und Funktionsaufrufen ermöglicht SCL eine sehr flexible und modulare Programmierung [?].

2.6 Grundlagen zu Inkrementalgebern

Inkrementalgeber erzeugen aus einer mechanischen Weg- oder Drehbewegung zwei zueinander um 90° phasenverschobene, rechteckförmige Ausgangssignale (A/B; Quadratur). Bei konstanter Geschwindigkeit entstehen gleichförmige Pulszüge; die Pulsfrequenz ist proportional zur Geschwindigkeit und kann über einen geeigneten Skalenfaktor in Geschwindigkeitswerte umgerechnet werden [?].

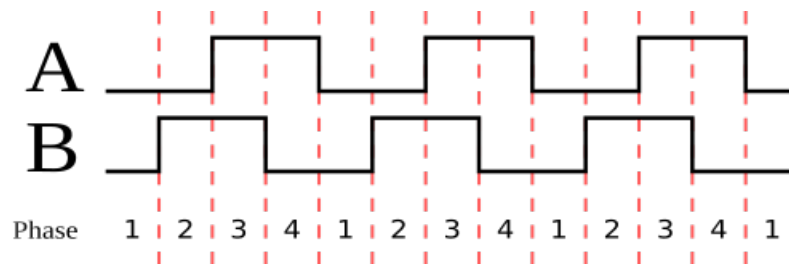


Abbildung 2.7: Zwei Rechteckwellen in Quadratur [?]

Rotative Geber realisieren dies typischerweise mit einer Lichtquelle, einer codierten Scheibe auf der Welle und Photodetektoren. Die Bewegungsrichtung wird aus der Phasenlage der Quadratur-Signale bestimmt: In einer Drehrichtung führt A vor B, in der anderen führt B vor A. Für eine höhere Auflösung kann nicht nur pro Puls, sondern auf jeder Flanke gezählt werden (1x-, 2x- oder 4x-Auswertung); dadurch lässt sich die Zählauflösung auf das bis zu Vierfache erhöhen [?, ?]. Aufgrund dieser Eigenschaften werden Inkrementalgeber häufig in Anwendungen eingesetzt, die eine präzise Messung und Regelung von Position und Geschwindigkeit erfordern.

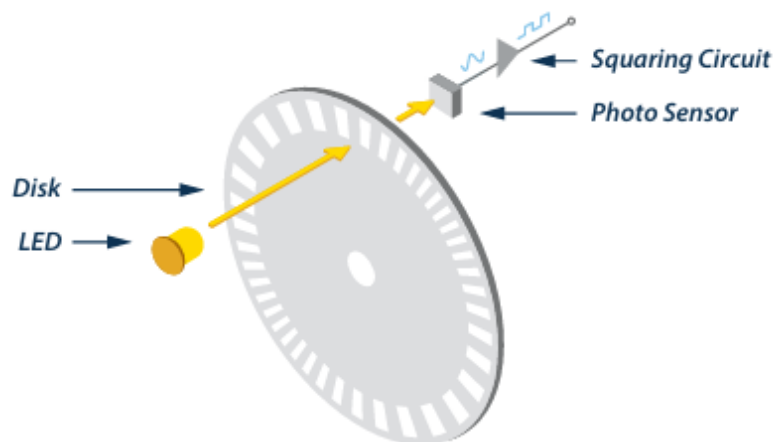


Abbildung 2.8: Funktionsweise eines Inkrementalgebers [?]

3 Analyse der Ablösung des FM350-2 durch das TM FAST Modul

In diesem Abschnitt werden die technischen Grundlagen und Eigenschaften der beiden betrachteten Module, FM350-2 und TM FAST, vorgestellt. Es werden deren Aufbau, Funktionsweise, Schnittstellen und typische Einsatzgebiete erläutert. Ziel ist es, die relevanten Technologien und Unterschiede beider Systeme darzustellen, um die Basis für die spätere Analyse der Ablösung des FM350-2 durch das TM FAST Modul zu schaffen.

3.1 Funktionsweise des FM350-2

Die Funktionsbaugruppe FM 350-2 ist eine 8-kanalige Zählerbaugruppe mit integrierter Dosierfunktion für das Automatisierungssystem S7-300. Sie wird vor allem in Bereichen eingesetzt, in denen Signale gezählt und schnelle Reaktionen auf bestimmte Zählerstände notwendig sind. Besonders relevant ist die FM 350-2 für Anwendungen in der Fertigungs- und Automatisierungstechnik, wie beispielsweise in Verpackungs-, Sortier- und Dosieranlagen sowie bei der Drehzahlregelung und Überwachung von Gasturbinen.

Sie unterstützt einen maximalen Zählbereich von -2^{31} bis $+2^{31} - 1$ (entsprechend $-2\,147\,483\,648$ bis $+2\,147\,483\,647$) und erreicht je nach verwendeter Gebersignale eine maximale Eingangsfrequenz von bis zu 20 kHz pro Kanal [?].

Die Baugruppe ermöglicht verschiedene Betriebsarten: endloses, einmaliges sowie periodisches Zählen (jeweils vorwärts oder rückwärts), Frequenzmessung, Drehzahlmessung, Periodendauermessung sowie präzises Dosieren. Der Zählvorgang kann wahlweise über das Anwenderprogramm (Software-Tor) oder über externe Signale (Hardware-Tor) gestartet und gestoppt werden. Dabei können Zähl-, Tor- und Richtungssignale direkt an die Baugruppe angeschlossen werden.

Für jeden Zählkanal ist es möglich, einen Vergleichswert zu definieren. Im Dosiermodus lassen sich bis zu vier Vergleichswerte je Kanal verwenden. Wird der definierte Vergleichswert erreicht, kann automatisch ein zugeordneter Ausgang gesetzt oder zurückgesetzt werden, um direkt steuerungsrelevante Aktionen auszulösen oder Prozessalarme zu generieren.

Innerhalb der Betriebsarten „Einmalig zählen“, „Periodisch zählen“ und „Dosieren“ lassen sich zusätzlich Zählgrenzen innerhalb des maximalen Zählbereichs festlegen. Im Vorwärtsbetrieb beginnt die Zählung bei 0, der Endwert ist dabei frei wählbar. Im Rückwärtsbetrieb wird der Startwert definiert, während der Endwert bei 0 liegt.

Die FM 350-2 unterstützt pro Zählkanal bis zu vier Prozessalarme. Zwei davon können durch Flankenwechsel am Hardware-Tor ausgelöst werden, zwei weitere sind abhängig von der jeweils eingestellten Betriebsart. Im Dosierbetrieb können bis zu fünf spezifische Alarmerzeuger erzeugt werden.

Für die Zählerfunktion können Signale unterschiedlicher Gebertypen verwendet werden. Zulässig sind ausschließlich prellfreie Geber, darunter 24-V-Inkrementalgeber (Gegentakt oder P-Schalter), 24-V-Impulsgeber mit Richtungspegel, 24-V-Initiatoren ohne Richtungspegel (z. B. Lichtschranken oder BERO Typ 2) sowie NAMUR-Geber nach DIN 19234. Die Signaltypen können an den Eingängen in Vierergruppen zwischen 24-V- und NAMUR-Signalen konfiguriert werden. Zu beachten ist, dass an für NAMUR konfigurierte Eingänge keine Signalspannungen über 8,2 V angelegt werden dürfen. An den Tor- (logische Freigabeeingänge, die den Zählpfad öffnen oder sperren) und Richtungseingängen sind ausschließlich 24-V-Signale erlaubt.

Zum Schutz vor Störungen sind alle Eingänge mit einem einheitlichen EingangsfILTER (RC-Glied) mit einer Filterzeit von 50 µs versehen. Schnelle Reaktionen auf Zählereignisse sind durch digitale Ausgänge realisierbar, wobei pro Kanal ein Ausgang und im Dosiermodus bis zu vier Ausgänge verfügbar sind. Diese können zählerstandsabhängig oder über Steuerbits angesteuert werden.

Das Verhalten der Baugruppe bei einem CPU-STOP ist parametrierbar. Es kann festgelegt werden, ob die Betriebsart fortgeführt oder abgebrochen wird und ob die digitalen Ausgänge ihren aktuellen Zustand beibehalten, auf Ersatzwerte gesetzt oder abgeschaltet werden. Dabei ist besondere Vorsicht geboten, da Ersatzwerte

auch an nicht freigegebenen Ausgängen anliegen können, was unter Umständen zu gefährlichen Anlagenzuständen führt.

Auch bei Ausfall der Baugruppenversorgung zeigt die FM 350-2 ein differenziertes Verhalten. Wird sie über einen Standard-Rückwandbus betrieben, erkennt die CPU bei Spannungsverlust einen Peripherie-Zugriffsfehler, und die Baugruppe startet nach Spannungswiederkehr nicht automatisch neu. Beim Einsatz eines aktiven Rückwandbusses hingegen wird der CPU ein Ziehen-Alarm und bei Wiederkehr der Versorgungsspannung ein Stecken-Alarm gemeldet.

Zusätzlich verfügt die FM 350-2 über umfassende Diagnosefunktionen. Mögliche Fehlerzustände wie fehlerhafte Gebersversorgung (NAMUR), fehlende oder fehlerhafte Parametrierung, Zeitüberwachungsfehler (Watchdog), Drahtbruch oder Kurzschluss sowie der Verlust von Prozessalarmen können erkannt und gemeldet werden [?].

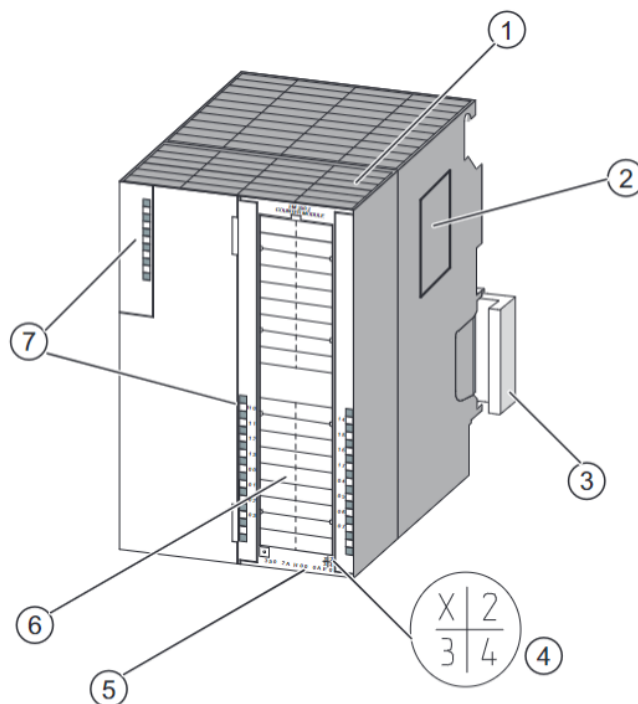


Abbildung 3.1: Hardware der FM-350-2 [?]

Nummer	Bezeichnung
1	Frontstecker
2	Typenschild
3	Busverbinder SIMATIC-Schnittstelle
4	Erzeugnisstand
5	Bestellnummer
6	Beschriftungsstreifen
7	Diagnose-LED
8	Status-LEDs

Tabelle 3.1: Kennzeichnungen und Komponenten der FM 350-2 [?]

3.1.1 Endloses Zählen bei der FM 350-2

Die Betriebsart „Endloses Zählen“ ermöglicht es der FM 350-2, Zählimpulse kontinuierlich zu erfassen, ohne dass der Zähler bei Erreichen einer Grenze stoppt. Wird beim Vorwärtszählen die obere Zählgrenze (+2 147 483 647) erreicht und ein weiterer Zählimpuls empfangen, springt der Zähler automatisch auf die untere Zählgrenze (−2 147 483 648) und zählt weiter. Analog dazu springt der Zähler beim Rückwärtszählen von der unteren Zählgrenze auf die obere Zählgrenze, sobald ein weiterer Zählimpuls erfolgt. Dadurch wird ein endloses Zählen innerhalb des 32-Bit-Zählbereichs realisiert.

Der Zählbereich ist fest auf 31 Bit vorzeichenbehaftet (−2 147 483 648 bis +2 147 483 647) und kann nicht verändert werden. Nach einem Neustart beginnt der Zähler bei 0.

Wurde ein Vergleichswert parametrierung, kann bei Erreichen des Vergleichswerts ein Prozessalarm ausgelöst und/oder ein Ausgang geschaltet werden.

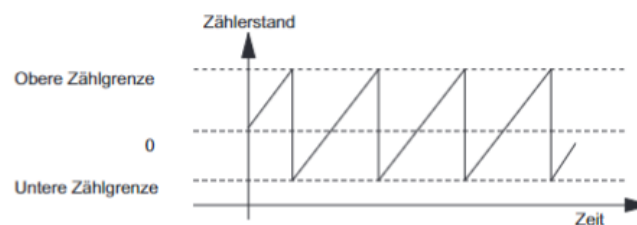


Abbildung 3.2: Endloses Zählen in Hauptzählrichtung vorwärts [?]

3.1.2 Einmaliges Zählen bei der FM 350-2

Die Betriebsart „Einmaliges Zählen“ ermöglicht das Erfassen einer festen Anzahl von Zählimpulsen zwischen einem definierten Start- und Endwert. Über die Parametrierung werden Startwert, Endwert und Hauptzählrichtung festgelegt.

Beim Zählen in Hauptzählrichtung vorwärts beginnt der Zähler bei 0 und zählt einmalig bis zum Endwert. Wird der Wert „Endwert-1“ erreicht und ein weiterer Zählimpuls empfangen, springt der Zähler zurück auf den Zählerstand 0 und bleibt dort stehen, auch wenn weitere Impulse folgen.

Beim Zählen in Hauptzählrichtung rückwärts startet der Zähler beim Startwert und zählt einmalig in Richtung 0. Erreicht der Zähler den Wert 1 und ein weiterer Zählimpuls erfolgt, springt er auf den Startwert zurück und bleibt stehen.

Wird entgegen der gewählten Hauptzählrichtung gezählt und dabei der Startwert unter- oder überschritten, meldet die Baugruppe den aktuellen Zählerstand mit negativem Vorzeichen zurück. Ein Überlauf oder Unterlauf findet in diesem Modus nicht statt.

Ist ein Vergleichswert parametrierbar, kann bei aktuellem Zählerstand = Vergleichswert ein Prozessalarm ausgelöst und/oder ein Ausgang geschaltet werden.

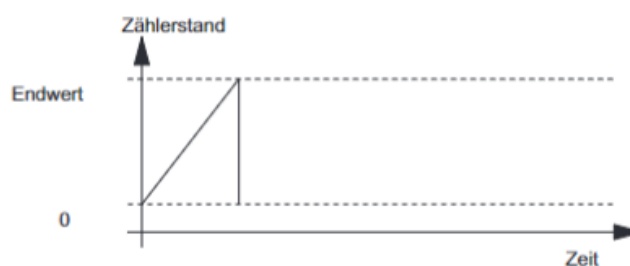


Abbildung 3.3: Einmaliges Zählen in Hauptzählrichtung vorwärts [?]

3.1.3 Periodisches Zählen bei der FM 350-2

Die Betriebsart „Periodisches Zählen“ erlaubt das Erfassen von Zählimpulsen innerhalb eines festgelegten Bereichs zwischen Startwert und Endwert. Nach Erreichen des Endwerts springt der Zähler automatisch auf den Startwert zurück und

zählt erneut bis zum Endwert. Dieses Verhalten wiederholt sich zyklisch, wodurch eine periodische Zählung realisiert wird.

Beim Zählen in Hauptzählrichtung vorwärts startet der Zähler beim Startwert (meist 0) und zählt bis zum Endwert. Sobald der Endwert erreicht ist und ein weiterer Zählimpuls erfolgt, springt der Zähler zurück auf den Startwert und beginnt erneut zu zählen. Die Zählimpulse werden kontinuierlich erfasst, sodass der Zählvorgang periodisch erfolgt.

Beim Zählen in Hauptzählrichtung rückwärts startet der Zähler am parametrierten Startwert und zählt in Richtung Endwert (meist 0). Wird der Endwert erreicht und ein weiterer Zählimpuls empfangen, springt der Zähler auf den Startwert zurück und zählt erneut rückwärts.

Wird entgegen der gewählten Hauptzählrichtung gezählt und dabei der Startwert unter- oder überschritten, meldet die Baugruppe den aktuellen Zählerstand mit negativem Vorzeichen zurück. Ein Überlauf oder Unterlauf findet in diesem Modus nicht statt; der Ausgangswert bleibt unverändert.

Ist ein Vergleichswert parametriert, kann bei aktuellem Zählerstand = Vergleichswert ein Prozessalarm ausgelöst und/oder ein Ausgang geschaltet werden.

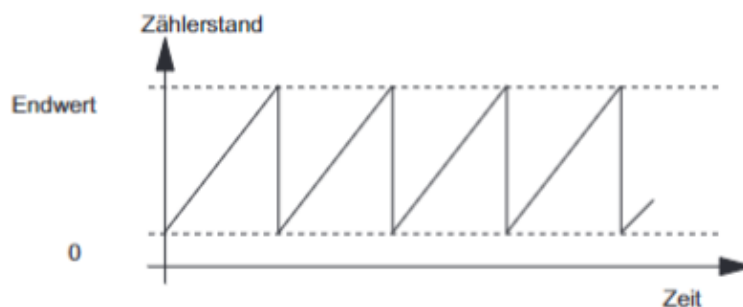


Abbildung 3.4: Periodisches Zählen in Hauptzählrichtung vorwärts [?]

3.1.4 Frequenzmessung mit der FM 350-2

Die FM 350-2 bietet eine integrierte Funktion zur Frequenzmessung, bei der die Anzahl der Impulse innerhalb eines parametrierbaren Zeitfensters gezählt wird. Die Zeitfensterlänge kann zwischen 10 ms und 10 s eingestellt werden. Nach Ablauf

des Zeitfensters wird die Frequenz als Anzahl der gezählten Impulse pro Sekunde berechnet und als Wert in Hz ausgegeben.

Die Frequenzmessung kann sowohl über das Anwenderprogramm als auch über externe Signale (Hardware-Tor) gestartet und gestoppt werden. Die Baugruppe unterstützt dabei verschiedene Vergleichswerte, sodass Prozessalarme ausgelöst werden können, wenn die gemessene Frequenz bestimmte Grenzwerte über- oder unterschreitet.

Die wichtigsten Merkmale der Frequenzmessung sind:

- Messbereich: 0 Hz bis 9.999.999 Hz (abhängig von Parametrierung und Signalart)
- Einstellbare Zeitfenster für die Messung (10 ms bis 10 s)
- Prozessalarme bei Über- oder Unterschreiten von Grenzwerten
- Start und Ende der Messung über Software- oder Hardware-Tor
- Frequenzvergleich mit parametrierten Sollwerten möglich

Die Frequenzmessung eignet sich besonders für Anwendungen, bei denen die Geschwindigkeit oder Drehzahl von Maschinen und Anlagen überwacht werden muss. Typische Einsatzfälle sind die Überwachung von Förderbändern, Drehtischen oder Antrieben.

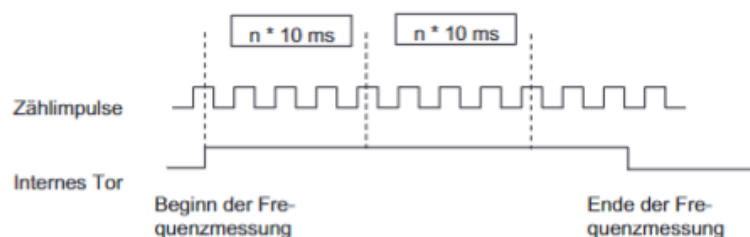


Abbildung 3.5: Prinzip der Frequenzmessung mit Zeitfenster bei der FM 350-2 [?]

3.1.5 Drehzahlmessung mit der FM 350-2

Die Drehzahlmessung ist eine spezielle Betriebsart der FM 350-2, die nahezu identisch mit der Frequenzmessung arbeitet. Neben der Länge des Zeitfensters muss bei

der Drehzahlmessung in der Parametermaske die Anzahl der Impulse pro Motor- oder Geberumdrehung angegeben werden.

Am Ende jedes Zeitfensters wird der Drehzahlwert aktualisiert. Die ermittelte Drehzahl wird in der Einheit 1×10^{-3} Umdrehungen/Minute ausgegeben.

Wird kein gültiger Wert ermittelt, wird -1 zurückgemeldet. Werden in einem Zeitintervall keine Impulse gezählt, liefert die Baugruppe 0×10^{-3} U/min (= 0 U/min).

Über zwei Drehzahlvergleichswerte (unterer und oberer Grenzwert) kann überwacht werden, ob die gemessene Drehzahl im vorgegebenen Bereich liegt. Beim Verlassen dieses Bereichs kann ein Prozessalarm ausgelöst werden. Die FM 350-2 prüft, ob Drehzahlobergrenze $>$ Drehzahluntergrenze ist und meldet einen Parametrierungsfehler, falls dies nicht der Fall ist.

Die Drehzahlmessung wird über die Torfunktionen gestartet und beendet. Folgende Prozessalarme sind möglich:

- Beginn der Drehzahlmessung durch HW-Tor (positive Flanke)
- Ende der Drehzahlmessung durch HW-Tor (negative Flanke)
- Ende der Messwerterfassung (Integrationszeit ist abgelaufen)
- Über- bzw. Unterschreiten der Drehzahlgrenzen

Die Drehzahlmessung eignet sich besonders für Anwendungen, bei denen die Drehzahl von Maschinen oder Antrieben überwacht und geregelt werden muss, wie z.B. bei Förderbändern, Drehtischen oder Motoren.

3.1.6 Periodendauermessung mit der FM 350-2

Bei sehr kleinen Frequenzen wird anstelle der Frequenz die Periodendauer gemessen. Im Betriebsart „Periodendauermessung“ erfasst die FM 350-2 die exakte Zeit zwischen zwei steigenden Flanken eines Eingangssignals. Die Messung wird über Torsignale (Hardware- oder Software-Tor) gestartet und beendet.

Die Periodendauer kann nur in der eingestellten Hauptzählrichtung erfasst werden. Der zulässige Messbereich liegt zwischen $40 \mu\text{s}$ und 120 s (entspricht $25\,000 \text{ Hz}$ bis $0,00833 \text{ Hz}$). Liegt kein gültiger Wert vor, wird -1 zurückgemeldet.

Über die Parametermaske können zwei Periodendauer-Grenzwerte eingestellt werden:

- Untere Grenze: 0 μs bis 119 999,999 μs
- Obere Grenze: 40 μs bis 120 000,000 μs

Folgende Prozessalarme sind möglich:

- Beginn der Periodendauermessung durch HW-Tor (positive Flanke)
- Ende der Periodendauermessung durch HW-Tor (negative Flanke)
- Ende der Messwerterfassung (Integrationszeit ist abgelaufen)
- Über- bzw. Unterschreiten der Periodendauergrenzen

Die Periodendauermessung eignet sich besonders für Anwendungen mit sehr niedrigen Frequenzen, bei denen eine präzise Zeitmessung zwischen zwei Ereignissen erforderlich ist, z.B. bei langsamen Bewegungen oder Taktgebern.

3.1.7 Dosierfunktion der FM 350-2

Die FM 350-2 bietet eine integrierte Dosierfunktion, mit der präzise Mengen von Materialien oder Teilen gesteuert und abgegeben werden können. Im Betriebsmodus „Dosieren“ werden jeweils vier Zählkanäle zu einem Dosierkanal zusammengefasst. Für jeden Dosierkanal können bis zu vier Vergleichswerte festgelegt werden, die einzeln oder gruppenweise verändert werden können.

Der aktuelle Zählerstand wird kontinuierlich mit den Vergleichswerten verglichen. Sobald der Zählerstand einen Vergleichswert erreicht, kann ein zugeordneter Digitalausgang angesteuert und/oder ein Prozessalarm ausgelöst werden. So lassen sich bis zu vier Dosierabschnitte innerhalb eines Dosierzyklus realisieren.

Die Dosierfunktion unterstützt sowohl den Vorwärts- als auch den Rückwärtsbetrieb. Im Rückwärtsbetrieb wird ein Startwert vorgegeben und der Zähler zählt bis zum Endwert (meist 0) herunter. Die Vergleichswerte definieren die einzelnen Dosierabschnitte, in denen jeweils ein Ausgang geschaltet oder ein Alarm ausgelöst werden kann.

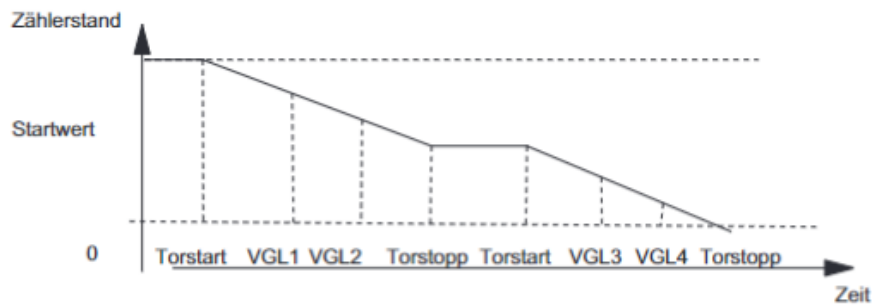


Abbildung 3.6: Dosierfunktion: Hauptzählrichtung rückwärts mit vier Vergleichswerten und Prozessalarmen [?]

Mögliche Prozessalarme im Dosierbetrieb sind:

- Beginn der Dosierung durch Hardware-Tor (positive Flanke)
- Abbruch/Unterbrechung der Dosierung durch Hardware-Tor (negative Flanke)
- Prozessalarm für jeden Vergleichswert (bis zu vier pro Kanal)
- Erreichen der Zählbereichsgrenzen (Endwert/Startwert)

Die Dosierfunktion eignet sich besonders für Anwendungen, bei denen exakte Mengen abgegeben werden müssen, wie z.B. in Abfüllanlagen, Mischprozessen oder bei der Steuerung von Förderbändern. Durch die flexible Parametrierung der Vergleichswerte und die schnelle Reaktion auf Zählereignisse können komplexe Dosieraufgaben zuverlässig und präzise umgesetzt werden.

3.1.8 Torfunktionen der FM 350-2

Viele Anwendungen erfordern, dass der Zählvorgang erst zu einem definierten Zeitpunkt, abhängig von anderen Ereignissen, gestartet oder gestoppt wird. Dieses Starten und Stoppen des Zählvorgangs geschieht bei der FM 350-2 über eine Torfunktion. Wird das Tor geöffnet, können Zählimpulse zum Zähler gelangen, der Zählvorgang wird fortgesetzt. Wird das Tor geschlossen, können keine Zählimpulse mehr zum Zähler gelangen, der Zählvorgang ist gestoppt.

Software-Tor und Hardware-Tor Die FM 350-2 besitzt zwei Torfunktionen:

- **Software-Tor (SW-Tor):** Das Software-Tor wird durch das Steuerbit `SW_GATE` im Anwenderprogramm gesetzt oder zurückgesetzt. Es kann ausschließlich durch einen Flankenwechsel von 0 auf 1 geöffnet und von 1 auf 0 geschlossen werden.
- **Hardware-Tor (HW-Tor):** Das Hardware-Tor wird durch ein externes Signal an einem digitalen Eingang der Baugruppe gesteuert. Es wird durch einen Flankenwechsel des Eingangssignals geöffnet oder geschlossen.

Internes Tor Das interne Tor ergibt sich als logische UND-Verknüpfung von HW-Tor und SW-Tor. Ist eines der Tore geschlossen, bleibt das interne Tor ebenfalls geschlossen und der Zählvorgang ist inaktiv. Nur wenn beide Tore offen sind, ist das interne Tor aktiv und der Zählvorgang läuft.

HW-Tor	SW-Tor	internes Tor	Zählvorgang
offen	offen	offen	aktiv
offen	geschlossen	geschlossen	inaktiv
geschlossen	offen	geschlossen	inaktiv
geschlossen	geschlossen	geschlossen	inaktiv

Tabelle 3.2: Zusammenhang der Torfunktionen und des Zählvorgangs [?]

Beispiel Mit dem Setzen des Torsignals wird das Tor geöffnet und die Zählimpulse werden gezählt. Wird das Torsignal weggenommen, wird das Tor geschlossen und die Zählimpulse werden nicht mehr vom Zähler erfasst. Der Zählerstand bleibt konstant.

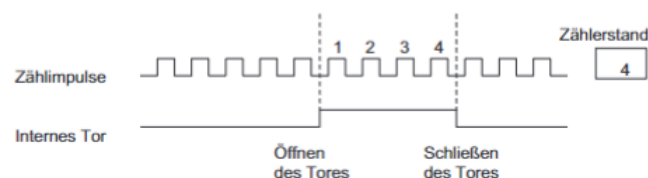


Abbildung 3.7: Öffnen und Schließen eines Tores und das Zählen der Impulse [?]

3.1.9 Prozessalarme der FM 350-2

Im Betrieb der Zählerbaugruppe FM 350-2 können verschiedene Fehler auftreten. Diese entstehen z. B. durch Defekte auf der Baugruppe, fehlerhafte Bedienung/-Verdrahtung oder widersprüchliche Parametrierung. Zur Erkennung und gezielten Reaktion unterscheidet die Baugruppe Fehlerklassen und löst bei Bedarf Prozessalarme aus.

Fehlerklassen und -arten

- Interner Fehler: Defekte oder fehlerhafte Zustände innerhalb der Baugruppe, z. B. Ansprechen der Zeitüberwachung (Watchdog).
- Externer Fehler: Probleme in der Peripherie außerhalb der Baugruppe, die keinem einzelnen Kanal zugeordnet werden können.
- Externer Kanalfehler: Kanalspezifische Fehler, z. B. Signalfehler durch einen NAMUR-Geber.
- Datenfehler: Unzulässige Aufträge von Automatisierungsgerät (AG) oder Programmiergerät (PG), z. B. Vergleichswerte außerhalb des Zählbereichs.

Reaktion der Baugruppe

- Interne, externe und externe Kanalfehler: Sammelfehler-LED (SF) leuchtet; es können Diagnosealarme ausgelöst werden (wenn freigegeben).
- Datenfehler: Auftrag wird abgelehnt; Eintrag im Diagnosepuffer; Behebung durch erneute Übertragung mit korrigierten Daten.

Sammelfehler-LED (SF) Leuchtet rot bei internem Baugruppenfehler, fehlerhafter Parametrierung oder Leitungsfehlern. Typische Ursachen: angesprochene Zeitüberwachung, verlorener Prozessalarm, fehlende/fehlerhafte Parametrierung, Probleme der Geberversorgung oder fehlerhafte NAMUR-Gebersignale (z. B. Drahtbruch/Kurzschluss).

Diagnosealarme und -daten Diagnosealarme können in der Parametrierung freigegeben werden. Beim Fehler ruft die CPU den Diagnose-OB OB 82 auf; das zyklische Programm wird unterbrochen.

- Diagnosedaten DS0: werden beim Aufruf von OB 82 automatisch übertragen.
- Diagnosedaten DS1: detaillierte Kanalinfos, auslesbar über FC DIAG_RD.
- Inhalt: Hinweise zu intern/extern/Parametrier-/Kanalfehlern, Ansprechen der Zeitüberwachung, verlorene Prozessalarme, Leitungs-/Geberversorgungsfehler.
- Zusätzlich können über SFC 52 benutzerdefinierte Meldungen in den Diagnosepuffer der CPU geschrieben werden.

Datenfehler und Quittierung Ergeben sich aus unzulässigen Aufträgen durch PG oder über FC CNT2_WR bzw. FB CNT2WRPN. Die Baugruppe prüft und lehnt fehlerhafte Aufträge ab; das Bit DATA_ERR im Zähler-Datenbaustein wird gesetzt. Behebung durch Korrektur und erneute Übertragung des Auftrags. Anzeige im Diagnosepuffer sowie im Menü Test > Fehlerauswertung.

3.2 Eigenschaften und Aufbau des TM FAST

Das TM FAST ermöglicht die Integration eigener Anwendungen in das SIMATIC S7-1500 Technologiemodul. Hardwareseitig besteht es aus einem Backend-Baustein, der die Siemens TM FAST-Firmware bereitstellt, und einem Frontend-Baustein, dem Intel® Cyclone 10 LP FPGA, auf dem die TM FAST-Anwendung läuft. Diese Anwendung definiert die Hardwarefunktionalität des Moduls und kann durch applikationsspezifische FPGA-Programmierung mittels Intel® Quartus® Prime angepasst werden [?, ?].

Eine ähnliche Baugruppe ist der High Speed Boolean Processor FM352-5 des SIMATIC S7-300 Systems, der mit einer Zykluszeit von 1 μ s schnelle Binärsteuerungen ermöglicht [?]. Das TM FAST FPGA bietet jedoch eine wesentlich höhere Flexibilität, erlaubt Zykluszeiten bis 20 ns und die Erzeugung präziser Pulsmuster. Somit können nicht nur digitale Signale, sondern auch komplexere Aufgaben wie Zählvorgänge, präzise Dosierungen oder pixelgenaue Punktmuster verarbeitet werden [?].

- Positionsabhängige Nocken
- Schnelle Reaktionen auf digitale Ereignisse
- Zählen von Signalen bis 5 MHz
- Auswertung von Inkremental- und Absolutwertgebern mit sofortiger Reaktion
- Präzise Steuerung von Lasern
- Pixelgenaue Punktmuster auf parallelen Spuren
- Exakte Dosieraufgaben
- Berechnungen und Vorverarbeitungen im Modul (z. B. Fehlerfrüherkennung)

Die freie Programmierbarkeit ermöglicht die Kombination und Anpassung einzelner Funktionen. Das TM FAST eignet sich daher besonders für Anwendungen, bei denen die SPS mit Peripheriemodulen nicht ausreichend schnell ist oder spezifische Einzelanwendungen erfordert.

Aufbau des TM FAST

Das TM FAST besteht aus einem Backend-Gerät, einem Frontend-Gerät, einer Prozessschnittstelle sowie der Anbindung an CPU und Peripherie (Abb. ??).

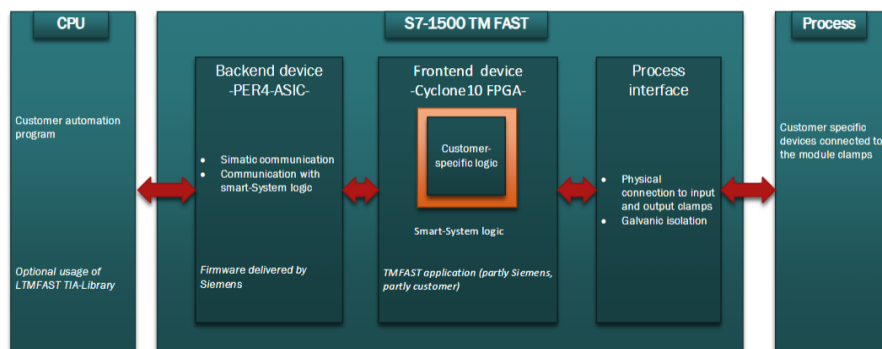


Abbildung 3.8: Blockdiagramm des TM FAST [?]

Backend-Gerät: Das Backend sorgt für die Kommunikation des TM FAST mit der S7-1500 über den Rückwandbus, regelt zyklischen und azyklischen Datentransfer, überwacht den Diagnosestatus und lädt die TM FAST-Anwendung auf

das FPGA. Zudem kontrolliert es die Systemparameter, die Modultemperatur und die Statusanzeigen [?].

Frontend-Gerät: Im Frontend läuft die anwendungsspezifische TM FAST-Software auf dem Intel® Cyclone 10 LP FPGA. Die Logik ist in zwei Teile unterteilt: FAST-Systemlogik (TFL FAST SYS) und Benutzerlogik (TFL FAST USER). Die Programmierung erfolgt in VHDL über Top-Entity-Strukturen, Architekturen und Bibliotheken. Über diese Schnittstellen können Ein- und Ausgänge gesteuert sowie die Kommunikation mit der CPU realisiert werden (Abb. ??).

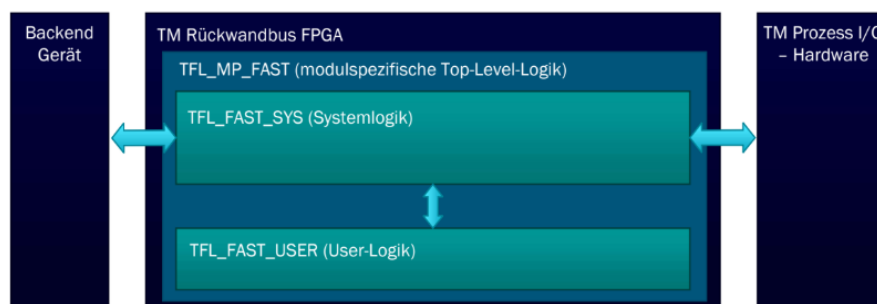


Abbildung 3.9: Aufbau von Frontend- und Backend-Gerät und Einbindung in das System [?]

Das FPGA-Modell 10CL025YU256A7G verfügt über 24 624 Logikelemente, 66 M9K Memory-Blöcke, 66 18×18-Bit-Multiplizierer, vier PLLs, 20 Clock-Leitungen und maximal 150 I/Os (Tab. ??) [?]. Jeder Memory-Block bietet 9 Kbit Speicher, die als RAM, FIFO oder ROM genutzt werden können. Die PLLs sind dynamisch rekonfigurierbar, und die GPIOs bieten Pull-Up-Widerstände und programmierbare Anstiegszeiten.

Ressource	Anzahl
Logikelemente	24 624
M9K Memory Blöcke	66
18×18 Multiplizierer	66
PLL	4
Clocks	20
Maximale I/O	150

Tabelle 3.3: Ressourcen des Intel® Cyclone 10 LP FPGA [?]

Äußerer Aufbau: Das Modul ist als Baugruppe der Bauform MP mit einem 40-poligen Frontstecker und vier zusätzlichen Klemmen zur Spannungsversorgung ausgeführt (Abb. ??). Ein TM FAST Debug Connector ermöglicht Service- und Entwicklungszugriffe unabhängig vom Frontstecker [?].

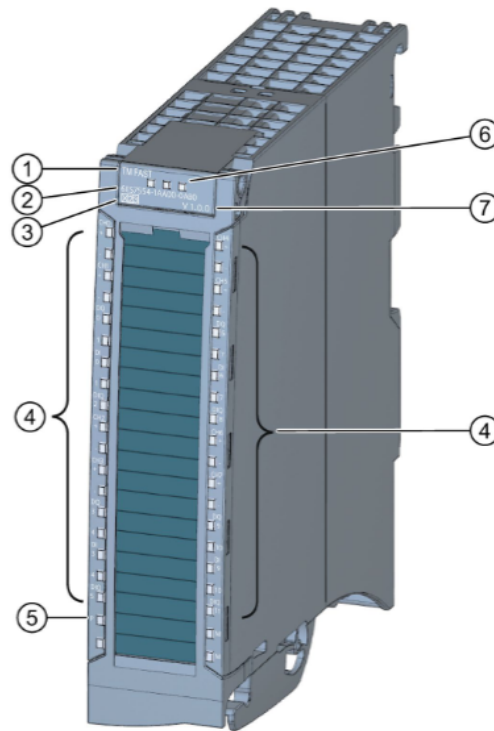


Abbildung 3.10: Technologiemodul TM FAST [?]

Das Modul bietet je acht 24 V-Digitaleingänge (DI) und -ausgänge (DQ) mit 0,1 A Ausgangsstrom sowie vier DIQ-Kanäle mit 0,3 A und maximal 1 kHz. Zudem stehen acht RS422/485-Kanäle bereit, die auch als TTL-Signale genutzt werden können. Eingangsverzögerungen für Digitaleingänge und RS422/485/TTL-Kanäle lassen sich sowohl fest parametrieren als auch über die TM FAST-Anwendung programmieren.

Kommunikation: Die Datenübertragung zwischen CPU und TM FAST kann zyklisch oder azyklisch erfolgen. Zyklische Ein- und Ausgangsdaten bestehen aus acht Doppelwörtern (32 Bit je Doppelwort). Die CPU-Schnittstelle betrachtet die Steuer- (CTRL IF) und Rückmeldedaten (FB IF). Azyklische Datenmengen können je nach Anwendungslogik konfiguriert werden (4, 32, 64 oder 128 Byte) und über die TIA Portal-Befehle `LTMFAST UserWriteREC` bzw. `LTMFAST UserReadREC` gelesen bzw. geschrieben werden [?, ?].

3.3 Direkter Vergleich der TM Fast und FM 350-2

Kriterium	TM FAST	FM 350-2
Unterschiede		
Architektur	Intel Cyclone 10 LP FPGA, frei programmierbar (VHDL), Backend/Frontend-Konzept	Feste ASIC-basierte Zählerbaugruppe, keine freie Hardwareprogrammierung
Zählerkanäle	Bis zu 14 Kanäle (DI, DQ, DIQ, RS422/485, TTL), flexibel konfigurierbar	8 Kanäle, fest als Zähler ausgelegt
Zählsignale / Gebereingänge	24 V-DI/DIQ, RS422/485, TTL, flexibel konfigurierbar, max. 5 MHz (DIQ/RS422/485), 200 kHz (DI/DQ)	Namur (DIN 19234), 24 V Impulsgeber, Inkrementalgeber, Richtungssignal, max. 20 kHz (A0...7), 10 kHz (B0...7)
Digitaleingänge	8x 24 V-DI, 4x 24 V-DIQ (konfigurierbar), max. 200 kHz (DI), 1 kHz (DIQ), Eingangsverzögerung parametrierbar (0–20 ms)	8x 24 V-DI, 0-Signal: -3 bis 5 V, 1-Signal: 11–30,2 V, Eingangsstrom: 0-Signal ≤ 2 mA, 1-Signal 9 mA, Eingangsverzögerung max. 50 μ s
Digitalausgänge	8x 24 V-DQ (0,1 A, 200 kHz), 4x DIQ (0,3 A, 1 kHz), High-/Low-aktiv, Überlast-/Kurzschlusschutz	Bis zu 4 Ausgänge pro Kanal (Dosiermodus), 24 V, direkt an Zählerereignisse gekoppelt, Verhalten parametrierbar
Reaktionszeit	$< 1 \mu$ s (laut Gerätehandbuch)	Nicht explizit angegeben
Zählbereich	32 Bit, flexibel nutzbar	32 Bit, fest (-2^{31} bis $2^{31} - 1$)
Betriebsarten	Frei programmierbar (z.B. Rundenzähler, Pulsfolgen, Dosierlogik, benutzerdefiniert)	Endlos, einmalig, periodisch, Dosieren, Frequenz-, Drehzahl-, Periodendauermessung
Parametrierung	Azyklisch und im laufenden Betrieb über UserWriteREC, flexible Steuerung	Parametrierung über STEP 7, Änderungen meist nur im Stopp-Betrieb
Ausgangslogik	Pro Kanal High-/Low-aktiv, Steuerbits, Rückmeldung, flexible Reaktion	Feste Zuordnung, Ausgänge direkt an Zählerereignisse gekoppelt

Diagnose	Umfangreiche Diagnosemöglichkeiten, frei programmierbar	Umfassende Diagnose, fest integriert (z.B. SF-LED, OB82, Diagnosedaten)
Integration	S7-1500, TIA Portal, flexible Einbindung, Peripherieanbindung	S7-300, STEP 7, klassische Integration
Gemeinsamkeiten		
Zählfunktionen	Hohe Zählauflösung (32 Bit), schnelle Verarbeitung, Einsatz als Zähler in der Automatisierungstechnik	
Frequenz-/Drehzahlmessung	Unterstützung von Frequenz- und Drehzahlmessung, Prozessüberwachung	
Industrielle Anwendungen	Verpackung, Sortierung, Dosierung, Maschinenbau, Prozessautomatisierung	
Diagnose	Umfangreiche Diagnose- und Fehleranzeigen, Unterstützung von Alarmen	

Tabelle 3.4: Vergleich der Eigenschaften von TM FAST und FM 350-2, insbesondere Zählsignale und Digitalein-/ausgänge (Zeitdaten TM FAST gemäß [?, ?], FM 350-2 gemäß Handbuch)

4 Entwicklung des VHDL-basierten Applikationsbeispiels

4.1 Anforderungen an die Neuentwicklung

Für die Neuentwicklung des Zählermoduls auf Basis des TM FAST ergeben sich verschiedene Anforderungen, die sich aus den funktionalen und technischen Zielen ableiten. Das Modul soll mindestens acht unabhängige Zählerkanäle, die Anzahl der unabhängigen Zählerkanäle des FM 350-2, bereitstellen, die flexibel als Digitaleingänge, Digitalausgänge oder RS485/TTL-Kanäle genutzt werden können. Jeder Kanal arbeitet mit einer ausreichend breiten Zählvariable, typischerweise 32 Bit, um auch große Wertebereiche abzudecken. Die Übergabe von Sollwerten und Parametern muss effizient und im laufenden Betrieb möglich sein, sodass der Prozess nicht unterbrochen wird. Verschiedene Betriebsmodi wie Rundenzähler, Reset, Hold und Start/Stop-Funktionen sind zu unterstützen. Für jeden Kanal werden Statusinformationen und Prozesswerte, etwa der aktuelle Zählerstand oder die Geschwindigkeit, rückgemeldet. Fehler müssen erkannt, sicher behandelt und an die Steuerung gemeldet werden. Die Lösung ist so zu gestalten, dass sie vollständig mit dem TIA Portal integrierbar und den Funktionsumfang der FM350-2 hat. Zudem ist auf eine ressourcenschonende Implementierung zu achten, um FPGA-Ressourcen effizient zu nutzen und eine einfache Erweiterbarkeit zu ermöglichen.

Die konkrete Umsetzung dieser Anforderungen wird in den folgenden Abschnitten detailliert beschrieben.

4.2 Modularisierung und Wiederverwendbarkeit

Die Architektur des VHDL-Designs ist konsequent modular aufgebaut und nutzt Generics, Arrays und Generate-Statements, um eine hohe Wiederverwendbarkeit und Erweiterbarkeit zu gewährleisten.

Ein Beispiel aus dem Code:

Listing 4.1: Array-Struktur und Generate-Schleife zur Kanalinstanziierung

```

1 type t_enc_count_array is array (0 to CHANNEL_COUNT-1)
2   of std_logic_vector(31 downto 0);
3 signal enc_count, upper_limit, next_upper_limit : t_enc_count_array
4 := (others => (others => '0'));
5 ...
6 channel_gen : for i in 0 to CHANNEL_COUNT-1 generate
7   ...
8 end generate;
```

So wird z.B. die Anzahl der Kanäle über das Generic ‘CHANNEL_COUNT‘ zentral festgelegt. Für alle kanalbezogenen Signale und Zustände werden Array-Typen verwendet, sodass die Logik für jeden Kanal identisch und unabhängig voneinander instanziiert werden kann.

Durch die Verwendung von Generate-Blöcken (‘channel_gen : for i in 0 to CHANNEL_COUNT-1 generate‘) wird die komplette Zähler- und Frequenzmesslogik pro Kanal automatisch erzeugt. Neue Kanäle können einfach durch Erhöhen von ‘CHANNEL_COUNT‘ und Anpassen der Mapping-Arrays hinzugefügt werden, ohne dass das Gesamtde-sign oder die Prozesslogik angepasst werden muss.

Im Code werden diese Arrays wie folgt deklariert und verwendet:

Listing 4.2: Deklaration und Verwendung der Mapping-Arrays für Kanalzuordnung

```

1 type t_channel_map is array (0 to 13) of integer;
2 constant CHANNEL_MAP_IN_CH : t_channel_map :=
3 (0, 1, 3, 4, 6, 7, 9, 10, 0, 1, 2, 3, 4, 5);
4 constant CHANNEL_MAP_OUT_CH : t_channel_map :=
5 (0, 1, 3, 4, 6, 7, 9, 10, 2, 5, 8, 11, 6, 7);
```

Auch die Kanalzuordnung (z.B. für unterschiedliche Hardware-Konfigurationen) erfolgt über Mapping-Arrays (‘CHANNEL_MAP_IN_CH‘, ‘CHANNEL_MAP_OUT_CH‘),

die flexibel anpassbar sind. Die Modularisierung erleichtert zudem die Wartung, das Testen und die Erweiterung um neue Funktionen, wie zusätzliche Diagnosemechanismen oder alternative Betriebsmodi, ohne tiefgreifende Änderungen an der bestehenden Architektur.

4.3 Ressourcenoptimierung

Das VHDL-Design nutzt gezielt Arrays und Bitfelder, um den Ressourcenverbrauch im FPGA zu minimieren. Die Anzahl der tatsächlich verwendeten Kanäle wird zentral über das Generic ‘CHANNEL_COUNT‘ festgelegt (Lst. ??). Alle kanalbezogenen Signale (wie Zählerstände, Statusflags, Steuerbits) werden als Arrays der Länge ‘CHANNEL_COUNT‘ deklariert. Dadurch werden nur so viele Flip-flops und Logikelemente instanziiert, wie für die aktivierten Kanäle tatsächlich benötigt werden.

Nicht genutzte Kanäle belegen somit keine unnötigen Ressourcen, da die Generate-Blöcke und Prozesse ausschließlich für die Indizes ‘0‘ bis ‘CHANNEL_COUNT-1‘ erzeugt werden. Auch die Kanaluordnung erfolgt über Mapping-Arrays, sodass keine redundante Logik für ungenutzte Ein-/Ausgänge entsteht.

Auch die Steuer- und Feedbackschnittstellen werden als Bitfelder (‘std_logic_vector‘) organisiert, sodass mehrere Steuer- und Statusinformationen platzsparend in einem einzigen Datenwort übertragen werden können. Dies reduziert den Bedarf an separaten Signalen und vereinfacht die Schnittstellenanbindung.

Im Fehlerfall oder bei CPU_STOP werden alle Status- und Ausgangssignale gezielt zurückgesetzt, sodass keine unnötigen Register oder Speicherbereiche dauerhaft belegt bleiben:

Listing 4.3: Gezieltes Rücksetzen von Signalen bei CPU_STOP zur Ressourcenoptimierung

```

1  if CPU_STOP = '1' and WR_REC_Control = x"00000000" then
2      upper_limit(i)      <= (others => '0');
3      next_upper_limit(i) <= (others => '0');
4      load_delay_cnt(i)   <= 0;
5      LimitReached(i)     <= '0';
6  end if;
```

Durch diese konsequente Nutzung von Arrays, Bitfeldern und Generate-Strukturen wird der FPGA optimal ausgelastet und der Ressourcenverbrauch auf das tatsächlich benötigte Minimum reduziert und für jeden nicht benutzten Kanal 370 Logikelemente (ca. 1.5%) eingespart.

WICHTIG. Bild der Anzahl von Logikelementen einfügen.

4.4 Fehler- und Diagnosestrategien

Das VHDL-Design implementiert eine systematische Fehlererkennung und sichere Fehlerbehandlung auf Kanalebene. Für jeden Kanal wird das Signal ‘system_fault(i)’ berechnet, das verschiedene Fehlerquellen überwacht:

- **DI_QI_BAD**: Fehler am zugeordneten Digitaleingang (z.B. Drahtbruch, Kurzschluss)
- **DQ_QI_BAD**: Fehler am zugeordneten Digitalausgang
- **RS485_QI_BAD**: Fehler an den RS485-Ein-/Ausgängen (z.B. Kommunikationsfehler, Pegelstörung)
- **LP_QI_BAD**: Fehler in der Spannungsversorgung (Low Power)

Die Fehler werden im Prozess ‘fault_dq_proc’ für jeden Kanal individuell ausgewertet. Sobald ein Fehler erkannt wird (‘system_fault(i) = ’1’), wird der entsprechende Ausgang (DQ oder RS485) automatisch in einen definierten Zustand (SAFE State) überführt. Der SAFE State ist abhängig von der konfigurierten Polarität und sorgt dafür, dass im Fehlerfall keine unkontrollierten Aktionen ausgelöst werden.

Die Fehlerdetails werden über das Feedback-Interface (z.B. ‘FB_IF7’) an die SPS rückgemeldet. Hierbei werden die Fehlerbits für DI, DQ, RS485 und die Kanalnummer gesetzt, sodass die Steuerung gezielt auf den Fehler reagieren kann. Ist kein Fehler vorhanden, werden die Fehlerbits explizit gelöscht.

Zusätzlich wird im Fehlerfall der Ausgang sofort deaktiviert und der Status ‘Start-DQ_active’ zurückgesetzt, um einen sicheren Anlagenzustand zu gewährleisten. Im Fall eines globalen CPU-Stop (z.B. durch ‘CPU_STOP = ’1’ und ‘WR_REC_Control

= x"00000000,,), werden alle Ausgänge ebenfalls in den SAFE State gesetzt und alle Statusregister zurückgesetzt.

Dieses Konzept stellt sicher, dass sowohl Einzelkanalfehler als auch globale Fehler jederzeit erkannt und sicher behandelt werden. Die SPS erhält über die Rückmeldebasis eine detaillierte Diagnose und kann gezielt Maßnahmen einleiten.

4.5 Synchronisation und Zustandsautomaten

Die Synchronisation der Ladevorgänge und die Überwachung der Zählergrenzen erfolgt im VHDL-Design über einen einfachen, aber effektiven Zustandsautomaten pro Kanal, der durch das Signal `load_delay_cnt(i)` realisiert wird. Dieser Zustandsautomat steuert, wann ein neuer Sollwert (Obergrenze) aus dem azyklisch übertragenen `WR_REC` übernommen wird und wie auf synchrone Steuerbefehle wie `Load` oder `StartDQ` reagiert wird.

Der Ablauf ist wie folgt:

- Bei einer Flanke auf `WR_REC_NEW` wird der neue Sollwert für den jeweiligen Kanal in `next_upper_limit(i)` zwischengespeichert.
- Erst wenn das synchrone Steuersignal `Load(i)` gesetzt wird, übernimmt der Zustandsautomat im Zustand `load_delay_cnt(i) = 0` den neuen Wert in `upper_limit(i)` und prüft sofort, ob der aktuelle Zählerstand bereits das Limit erreicht hat.
- Der Automat wechselt in den Zustand 1, solange `Load(i)` aktiv ist, und kehrt nach Loslassen des Signals wieder in den Grundzustand zurück.
- Das Limit-Flag `LimitReached(i)` wird gesetzt, sobald der Zählerstand die Obergrenze erreicht oder ein Überlauf erkannt wird. Mit dem Steuersignal `StartDQ(i)` kann dieses Flag zurückgesetzt und ein neuer Dosierzyklus gestartet werden.

Durch diese Trennung von asynchroner Sollwertübertragung (über `WR_REC`) und synchroner Steuerung (über `CTRL_IF/Load`) wird sichergestellt, dass Werteänderungen nur gezielt und deterministisch übernommen werden. Gleichzeitig verhindert der Zustandsautomat Race Conditions und garantiert, dass keine Sollwertänderung während eines laufenden Zyklus unbemerkt bleibt.

Ein Auszug aus dem VHDL-Code illustriert dieses Prinzip:

Listing 4.4: Zustandsautomat zur synchronisierten Sollwertübernahme und Limitüberwachung

```

1  process(CLK, RST)
2  begin
3      if RST = '1' then
4          upper_limit(i)      <= (others => '0');
5          next_upper_limit(i) <= (others => '0');
6          load_delay_cnt(i)   <= 0;
7          LimitReached(i)     <= '0';
8      elsif rising_edge(CLK) then
9          if WR_REC_NEW = '1' then
10             next_upper_limit(i) <=
11                 std_logic_vector(unsigned(WR_RECxx) - 1);
12         end if;
13         case load_delay_cnt(i) is
14             when 0 =>
15                 if Load(i) = '1' then
16                     upper_limit(i) <= next_upper_limit(i);
17                     load_delay_cnt(i) <= 1;
18                     if unsigned(enc_count(i)) >=
19                         unsigned(next_upper_limit(i)) then
20                         LimitReached(i) <= '1';
21                     else
22                         LimitReached(i) <= '0';
23                     end if;
24                 end if;
25             when 1 =>
26                 if Load(i) = '0' then
27                     load_delay_cnt(i) <= 0;
28                 end if;
29             when others =>
30                 load_delay_cnt(i) <= 0;
31         end case;
32         if StartDQ(i) = '1' then
33             LimitReached(i) <= '0';
34         end if;
35         if overflow(i) = '1' then
36             LimitReached(i) <= '1';
37         end if;
38     end if;
39 end process;

```

Dieses Konzept sorgt für eine robuste, deterministische und synchronisierte Steuerung aller Zähl- und Dosierprozesse im FPGA.

4.6 Zyklische und azyklische Kommunikation

Das VHDL-Design trennt klar zwischen zyklischer und azyklischer Kommunikation, um Prozesssicherheit und Flexibilität zu maximieren. Die gesamte Kommunikation erfolgt über drei Schnittstellen: CTRL_IF (zyklische Steuerung), FB_IF (zyklisches Feedback), WR_REC (azyklische Sollwertübertragung) und RD_REC (azyklisches Auslesen).

Die zyklischen Schnittstellen sind für zeitkritische Steuerungsaufgaben konzipiert und haben jeweils eine Datenmenge von 8 Doppelwörter mit jeweils 64 Bit. Bei einer Kanalanzahl von 14 bleibt dadurch nur 1 Word pro Kanal und 1 Doppelwort für zusätzliche Parameter übrig. Dies ist ausreichend für die Übertragung von Steuer- und Statusinformationen, jedoch nicht für umfangreiche Sollwertänderungen.

Dafür ist die azyklische Kommunikation geeignet, da Sollwerte nur selten im laufenden Prozess geändert werden sollen und die azyklische Kommunikation eine einstellbare Datenmenge hat. Sie wurde auf 64 Byte, also 16 Doppelwörter, eingestellt. Dadurch ist es möglich, alle Sollwerte der 14 Kanäle in einem einzigen azyklischen Transfer zu übertragen und noch 2 Doppelwörter für zukünftige Erweiterungen oder zusätzliche Parameter zu reservieren.

Zyklische Steuerung (CTRL_IF): Zyklische Steuerbefehle werden über die CTRL_IF-Schnittstelle übertragen und steuern in jedem SPS-Zyklus zentrale Funktionen wie Reset, Load, Hold, StartDQ oder EnableDQ für jeden Kanal. Diese Steuerbits werden im Prozess `ctrl_fb_mapping_proc` aus den CTRL_IF-Worten extrahiert und auf die internen Steuersignale der jeweiligen Kanäle gemappt. Dadurch ist eine deterministische, taktsynchrone Steuerung aller Kanäle gewährleistet.

Die einzelnen Steuerbits haben dabei folgende Funktionen:

- **CounterReset:** Setzt den Zähler des jeweiligen Kanals manuell zurück (Software-Reset für den Encoder).

- **EnableDQ:** Aktiviert den Digitalausgang des Kanals. Ist das Signal auf 0, bleibt der Ausgang in einem definierten Zustand (abhängig von der Polarity-Einstellung). Für den Normalbetrieb wirkt es mit **StartDQ_active** zusammen.
- **Load:** Löst das Laden des Sollwertes aus **UserDataContent** über den azyklischen Befehl **WR_REC** in den Zähler aus. Dabei wird überprüft, ob das neue Limit direkt erreicht wurde (falls der neue Sollwert kleiner als der aktuelle Zählerstand ist, wird der Zähler auf 0 gesetzt).
- **Hold:** Friert den aktuellen Zählerstand ein und verhindert, dass der Encoder weiter hoch- oder herunterzählt.
- **StartDQ:** Löscht das **LimitReached**-Latch und startet einen neuen Zähl- bzw. Ausgabzyklus. Der Ausgang bleibt aktiv, bis **StartDQ** erneut getriggert wird.
- **FBIF_Control:** Bestimmt die Rückmeldung an die SPS. Bei 1 wird die Frequenzmessung (**frequency_speed**, in Impulsen pro Sekunde) ausgegeben, bei 0 der Encoderzählerstand (**enc_count**).
- **Polarity:** Legt die Polarität des Ausgangs fest. Bei 0 bedeutet dies: aktiver Zustand = 1, sicherer Zustand = 0. Bei 1 kehrt sich dies um. Diese Einstellung wirkt sich sowohl auf den Normalbetrieb als auch auf den Fehlerfall aus.

Zyklisches Feedback (FB_IF): Das Feedback-Interface (FB_IF) liefert in jedem SPS-Zyklus aktuelle Status- und Prozesswerte für alle Kanäle. Jeder Kanal stellt ein 16-Bit-Wort bereit, dessen Bitstruktur wie folgt aufgebaut ist:

- **Bit 0–14:** Aktueller Prozesswert (z.B. Zählerstand oder Frequenz).
- **Bit 15:** Spiegelung des **Select_Feedback_Value**-Flags (Diagnose/Rückmeldung).
- **Bit 16 (MSB):** Status des Ventilausgangs (1 = Ausgang ein, 0 = Ausgang aus).

Zusätzlich werden über **FB_IF** Fehler- und Statussignale übertragen:

- **ErrorChannelNumber:** Nummer des letzten Kanals mit Fehler (0 = kein Fehler).

- **DI_DQ_BAD:** Bit-codierter Fehlerstatus für Digitaleingänge/-ausgänge.
- **RS485_LP_BAD:** Bit-codierter Fehlerstatus für RS485 oder L+.
- **StatusCode:** Applikations-Fehlercode und Aktivitätsanzeige.

Damit ist eine kontinuierliche Überwachung und Diagnose aller Kanäle durch die SPS möglich.

Listing 4.5: Minimalvariante der for-Schleife zur CTRL_IF–FB_IF Zuordnung

```

1  for i in 0 to CHANNEL_COUNT-1 loop
2      CounterReset(i) <= CTRL_IF0(24);
3      EnableDQ(i)      <= CTRL_IF0(26);
4      Load(i)          <= CTRL_IF0(27);
5      HoldSW(i)         <= CTRL_IF0(28);
6      StartDQ(i)        <= CTRL_IF0(29);
7      FBIF_Control(i) <= CTRL_IF0(30);
8
9      FB_IF0(29 downto 16) <= FBIF(i)(13 downto 0); -- Messwerte
10     FB_IF0(31) <= StartDQ_active(i);               -- Status
11     FB_IF0(30) <= FBIF_Control(i);                 -- Mode
12 end loop;
```

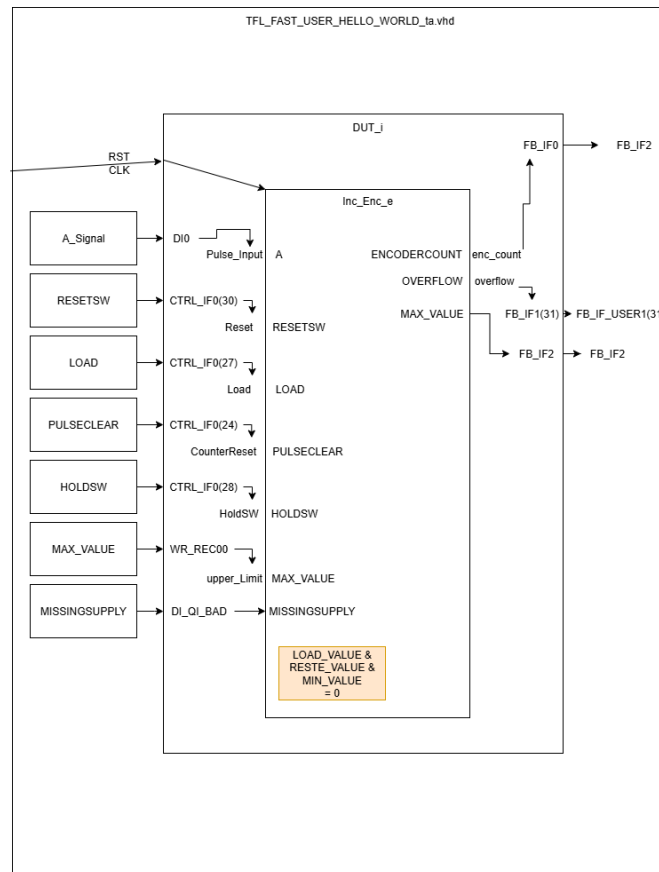


Abbildung 4.1: Diagram der einer Kanalstruktur

Azyklische Sollwertübertragung (WR_REC): Azyklische Sollwertübertragungen erfolgen über die WR_REC-Schnittstelle. Neue Obergrenzen (Sollwerte) für die Zählerkanäle werden als 32-Bit-Werte übertragen und im Prozesszyklus bei einer Flanke auf WR_REC_NEW in die Buffer-Variable `next_upper_limit(i)` übernommen. Erst wenn das zyklische Steuersignal `Load(i)` gesetzt wird, übernimmt der Kanal den neuen Sollwert in die aktive Obergrenze (`upper_limit(i)`). Dadurch wird verhindert, dass Sollwertänderungen während eines laufenden Dosierzyklus unbemerkt übernommen werden.

Dieses Zusammenspiel sorgt für hohe Prozesssicherheit: Azyklische Änderungen werden gepuffert und erst durch ein gezieltes, zyklisches Steuersignal wirksam. Gleichzeitig bleibt das System flexibel, da Sollwerte jederzeit aktualisiert werden können, ohne den laufenden Betrieb zu stören. Die klare Trennung der Kommunikationswege und die Synchronisation über Steuersignale verhindern Wettlaufschaltung (Race Conditions), das Auslösen von undefinierten Zuständen aufgrund

von Zeitverhalten und ermöglichen eine deterministische, fehlertolerante Steuerung aller Kanäle.

Desweiteren wird über den WR_REC das folgende Signal übertragen:

- **WR_REC_Control:** Globales Steuersignal, das das Verhalten bei CPU-Stopp festlegt. Bei x"00000000" wird ein *Abort Mode* aktiviert (Ausgänge sofort ausgeschaltet, alle Kanäle zurückgesetzt). In allen anderen Fällen läuft der *Continue Mode*, bei dem die Kanäle ihre aktuelle Operation fortsetzen.

Ein Auszug aus dem VHDL-Code illustriert das Prinzip:

Listing 4.6: Übernahme azyklischer Sollwerte aus WR_REC in die Kanalpuffer

```

1  if WR_REC_NEW = '1' then
2      next_upper_limit(i) <= std_logic_vector(unsigned(WR_RECxx) - 1);
3  end if;
4  ...
5  if Load(i) = '1' then
6      upper_limit(i) <= next_upper_limit(i);
7  end if;
```

Das Design trennt klar zwischen zyklischer (CTRL_IF) und azyklischer (WR_REC) Kommunikation. Zyklische Steuerbefehle steuern zentrale Funktionen wie Reset, Load, Hold, StartDQ oder EnableDQ. Azyklische Sollwertübertragungen werden gepuffert und erst durch ein zyklisches Steuersignal wirksam. Dadurch bleibt das System flexibel und prozesssicher. Jeder Kanal stellt ein 16-Bit-Wort bereit, das Prozesswert, Diagnose- und Statusinformationen enthält. Fehler- und Statussignale sind klar codiert und ermöglichen eine gezielte Diagnose durch die SPS.

4.7 Sicherheit bei CPU_STOP

Im Fehlerfall oder bei einem CPU_STOP muss das System in einen definierten Zustand übergehen, um unkontrollierte Aktionen und potenzielle Gefährdungen zu verhindern. Dies wird im VHDL-Design durch gezielte Abfragen und Rücksetzungen aller relevanten Steuer- und Ausgangssignale realisiert.

Im gezeigten Code wird der sichere Zustand insbesondere durch die Abfrage von CPU_STOP und WR_REC_Control = x"00000000" in mehreren Prozessen sichergestellt:

- **ctrl_fb_mapping_proc:** Sobald ein CPU_STOP erkannt wird und das Kontrollwort auf Null steht, werden für alle Kanäle die Steuerbits wie **EnabledQ**, **Load**, **HoldSW**, **StartDQ** und **CounterReset** explizit auf '0' bzw. '1' (Reset) gesetzt. Damit werden alle Ausgänge deaktiviert und alle Statusregister zurückgesetzt.
- **channel_gen:** Im FSM-Prozess für jeden Kanal werden im CPU_STOP-Fall die Obergrenzen, Puffer und Statusflags wie **upper_Limit**, **next_upper_limit**, **load_delay_cnt** und **LimitReached** auf Null bzw. Grundzustand zurückgesetzt.
- **fault_dq_proc:** Die Ausgangssignale (DQ) und der Status **StartDQ_active** werden im Fehlerfall oder bei CPU_STOP explizit auf SAFE-State ('0') gesetzt. Damit wird sichergestellt, dass keine Ausgänge mehr aktiv sind.

Ein Auszug aus dem Code illustriert das Verhalten:

Listing 4.7: Rücksetzen aller Ausgänge und Register im CPU_STOP-Fall

```

1   DQ(i)                <= '0';
2   StartDQ_active      <= (others => '0');
3   upper_Limit(i)      <= (others => '0');
4   next_upper_limit(i) <= (others => '0');
5   load_delay_cnt(i)   <= 0;
6   LimitReached(i)    <= '0';
7 end if;
```

Durch diese Maßnahmen wird garantiert, dass das System im Fehlerfall oder bei einem CPU_STOP deterministisch und zuverlässig in einen definierten Zustand übergeht. Dabei handelt es sich jedoch nicht um eine SIL-Klassifizierung oder funktionale Sicherheit im engeren Sinn, sondern vielmehr um die Gewährleistung eines stabilen Prozesses mit klar definierten Zuständen im Fehlerfall. Auf diese Weise werden ungewollte Schalthandlungen oder undefinierte Zustände verhindert. Im Gegensatz dazu sind für funktionale Sicherheit spezielle F-CPU's und Safety-Module vorgesehen, wie sie im Rahmen von SIMATIC Safety Integrated angeboten werden [?].

4.8 Flexibilität der Kanalzuordnung

Aufgrund des Designs von Dosieranlagen wird bei ihnen immer nur in eine Richtung abgefüllt. Daher muss bei Inkrementalgebern lediglich auf die A-Flanke, die Impulsflanke, geachtet werden, während das B-Ausgangssignal ignoriert werden kann. Dies ermöglicht, dass für jeden Kanal nur ein digitaler Eingang benötigt wird.

Das VHDL-Design unterstützt diese Struktur durch spezielle Mapping-Arrays, die eine flexible Zuordnung der Ein- und Ausgänge zu den einzelnen Kanälen erlauben. Die Arrays `CHANNEL_MAP_IN_CH` und `CHANNEL_MAP_OUT_CH` definieren für jeden logischen Kanal, welcher physikalische Eingang bzw. Ausgang genutzt wird. Auf diese Weise kann das Design problemlos an verschiedene Hardware-Konfigurationen oder Kundenanforderungen angepasst werden, ohne die eigentliche Kanal-Logik zu ändern.

Im Beispiel werden die ersten acht Kanäle als klassische DI/DQ-Kanäle betrieben, während die weiteren Kanäle flexibel als RS485 oder zusätzliche DIQ-Kanäle genutzt werden.

Standardmäßig werden die ersten acht Kanäle als klassische DI/DQ-Kanäle betrieben, während die weiteren Kanäle flexibel als RS485 oder zusätzliche DIQ-Kanäle genutzt werden können. Die Auswahl des Eingangstyps erfolgt dynamisch im Code.

Jeder Kanal greift über diese Mapping-Arrays auf die korrekten Ein- und Ausgänge zu:

```
1 input_selector(i) <= RS485_RX(CHANNEL_MAP_IN_CH(i)) when
2 i > 8 else DI(CHANNEL_MAP_IN_CH(i));
3 ...
4 DQ(CHANNEL_MAP_OUT_CH(i)) <= out_val;
```

Dadurch ist es möglich, die Kanalbelegung zentral zu ändern, etwa um bestimmte Kanäle als RS485, Digitaleingang oder Digitalausgang zu nutzen. Auch bei Änderungen der Hardware (z.B. andere Verdrahtung oder Erweiterung der Kanäle) muss lediglich das Mapping angepasst werden, ohne dass die restliche Kanal- oder Prozesslogik betroffen ist. Dies erhöht die Wartbarkeit und Wiederverwendbarkeit

des Designs erheblich und erlaubt eine schnelle Anpassung an neue Anforderungen.

4.9 Automatische Überlaufkompensation

Beim Verlassen auf Hardware kommt es stets zu einem zeitlichen Verzug, der durch Auswerten und physikalisches Schalten entsteht. Dies betrifft auch die Abfüllung von Flüssigkeiten, da trotz schneller Reaktionszeiten immer ein gewisser Verzug, bis das Ventil mechanisch oder hydraulisch vollständig geschlossen ist [?].

Um den beschriebenen Nachlaufeffekt zu kompensieren und eine präzise Dosierung zu gewährleisten, existieren verschiedene Lösungsansätze:

1. Prädiktion des Abschaltzeitpunkts

Der optimale Zeitpunkt für das Schließen des Ventils wird berechnet. Grundlage bilden die aktuelle Durchflussrate sowie die bekannte Ventil-Schließzeit. So lässt sich der Nachlauf berücksichtigen und die Zielmenge genauer erreichen.

2. Zweistufige Dosierung

Das Produkt wird zunächst mit hoher Geschwindigkeit bis auf etwa 90% der Zielmenge eingefüllt, anschließend erfolgt eine langsame, präzise Befüllung, um die gewünschte Endmenge exakt zu erreichen.

3. PID-geregelte Dosierung

Ein PID-Regler sorgt durch kontinuierliche Anpassung an die Abweichung zwischen Soll- und Ist-Wert für hohe Genauigkeit, insbesondere bei dynamischen Prozessen.

4. Massendurchflussmesser

Die direkte Messung der eingefüllten Masse ermöglicht besonders präzise Dosierung, da Schwankungen im Volumenstrom oder Materialeigenschaften automatisch berücksichtigt werden.

Implementierte automatische Überlaufkompensation

In der Implementierung wird eine automatische Überlaufkompensation realisiert, um die Dosiergenauigkeit trotz Nachlauf- und Verzögerungseffekten zu gewährleisten. Jeder Kanal besitzt einen 32-Bit-Zähler, dessen oberes Limit beim Start einer Dosierung aus den Sollwerten übernommen wird. Sobald der aktuelle Zählerwert das Limit erreicht oder überschreitet, wird das Signal `LimitReached` gesetzt und der Ausgang deaktiviert.

Nach Erreichen des Sollwertes wird der Zähler zurückgesetzt, und der Nachlauf, der weiterhin vom Encoder erfasst wird, wird auf den Zähler addiert. Somit ergibt sich bei der ersten Abfüllung:

$$V_{\text{total}} = V_{\text{target}} + V_{\text{current_overflow}}$$

In den nachfolgenden Zyklen gilt:

$$V_{\text{total}} = V_{\text{target}} + V_{\text{current_overflow}} - V_{\text{last_overflow}}$$

Mit konstantem Überlauf:

$$V_{\text{current_overflow}} = V_{\text{last_overflow}}$$

wird die Nachlaufmenge aus dem vorherigen Zyklus berücksichtigt und im nächsten Zyklus nicht erneut abgefüllt, da sie bei kontinuierlichem Abfüllen durch den Ventilnachlauf automatisch hinzugefügt wird. Dies ermöglicht eine konstante Dosiergenauigkeit.

Vor- und Nachteile der Überlaufkompensation

Die Überlaufkompensation bietet den Vorteil, dass sie die Systemträgheit, insbesondere die mechanische Nachlaufzeit des Ventils, berücksichtigt und so systematische Fehler durch den Nachlauf kompensiert. Unter konstanten Bedingungen wie gleichbleibendem Druck, konstanter Temperatur und gleichbleibender Viskosität arbeitet das Verfahren sehr genau. Außerdem wird keine zusätzliche Hardware

benötigt, und die Methode ist selbstlernend beziehungsweise adaptiv, da jeweils der zuletzt gemessene Überlaufwert für die nächste Dosierung verwendet wird. Dadurch eignet sich die Überlaufkompensation besonders gut für kontinuierliche Prozesse.

Nachteilig ist, dass bei schwankenden Bedingungen, beispielsweise Änderungen von Druck, Temperatur oder Viskosität, die Genauigkeit der Kompensation beeinträchtigt werden kann. Nach einem Neustart steht für die erste Dosierung kein Überlaufwert zur Verfügung. Störungen wie Luftblasen im System können zu falschen Überlaufwerten führen, und bei einem Produktwechsel muss das System zunächst neu eingelernt werden, um wieder präzise zu arbeiten. Bei stark unterschiedlichen Füllmengen kann es zudem erforderlich sein, mit verschiedenen Überlaufwerten zu arbeiten, um weiterhin eine genaue Dosierung sicherzustellen.

4.10 Erweiterbarkeit für zukünftige Anforderungen

Das vorgestellte VHDL-Design ist gezielt auf Erweiterbarkeit ausgelegt, um zukünftige Anforderungen wie zusätzliche Messmodi, neue Schnittstellen oder komplexere Diagnosemechanismen einfach integrieren zu können. Dies wird durch die konsequente Nutzung von Generics, Array-Typen und Generate-Blöcken erreicht. Die zentrale Konstante 'CHANNEL_COUNT' legt die Anzahl der Kanäle fest, sodass durch einfaches Erhöhen dieses Werts und Anpassen der Mapping-Arrays ('CHANNEL_MAP_IN_CH', 'CHANNEL_MAP_OUT_CH') weitere Kanäle hinzugefügt werden können, ohne die bestehende Logik zu verändern.

Neue Funktionen wie alternative Messverfahren (z.B. Frequenzmessung, Impulsbreitenmessung) lassen sich modular als zusätzliche Komponenten (wie 'FB82_Freq32_e') pro Kanal einbinden. Die Steuer- und Feedbackschnittstellen sind als Bitfelder und Arrays organisiert, sodass weitere Steuerbits oder Statusinformationen problemlos ergänzt werden können. Auch die Fehler- und Diagnoselogik ist pro Kanal gekapselt und kann durch zusätzliche Überwachungsmechanismen erweitert werden.

Durch die klare Trennung von Steuer-, Daten- und Diagnosepfaden sowie die Nutzung von Generate-Strukturen ist das Design flexibel anpassbar und für zukünftige

Erweiterungen vorbereitet. Dies ermöglicht eine nachhaltige Weiterentwicklung der Applikation, ohne dass grundlegende Architekturänderungen erforderlich sind.

Ein Auszug aus dem VHDL-Code zeigt die zentrale Rolle der Generics und Arrays für die Erweiterbarkeit:

```

1  constant CHANNEL_COUNT : natural := 8;
2  type t_enc_count_array is array (0 to CHANNEL_COUNT-1) of
3  std_logic_vector(31 downto 0);
4  signal enc_count, upper_limit, next_upper_limit : t_enc_count_array
5  := (others => (others => '0'));
6  ...
7  channel_gen : for i in 0 to CHANNEL_COUNT-1 generate
8  end generate;
```

4.11 Testbarkeit und Simulation

Die Architektur des VHDL-Designs ist gezielt auf eine hohe Testbarkeit und einfache Simulation ausgelegt. Durch die konsequente Trennung von Steuer- und Datenpfaden sowie die Verwendung von Arrays und Generate-Blöcken kann jeder Kanal unabhängig simuliert und getestet werden. Die zentrale Steuerung erfolgt über klar definierte Schnittstellen (CTRL_IF, WR_REC), sodass gezielte Stimuli für einzelne Kanäle oder Funktionen einfach im Testbench gesetzt werden können.

Im gezeigten Code werden alle kanalbezogenen Signale als Arrays deklariert, z.B.:

```

1  type t_enc_count_array is array (0 to CHANNEL_COUNT-1) of
2  std_logic_vector(31 downto 0);
3  signal enc_count, upper_limit, next_upper_limit : t_enc_count_array
4  := (others => (others => '0'));
```

Dadurch lassen sich im Test gezielt einzelne Kanäle ansprechen, ohne Seiteneffekte auf andere Kanäle zu verursachen.

Die Steuer- und Feedbackschnittstellen sind ebenfalls klar voneinander getrennt und als Bitfelder organisiert. Dies ermöglicht es, in der Simulation gezielt Steuerbits (z.B. Load, StartDQ, CounterReset) für einzelne Kanäle zu setzen und die Reaktion des Designs zu beobachten.

Die Verwendung von Generate-Blöcken wie

```
1 channel_gen : for i in 0 to CHANNEL_COUNT-1 generate
2     -- Kanal-Logik
3 end generate;
```

erlaubt es, die Logik für jeden Kanal separat zu simulieren und zu validieren. Fehlerfälle, Grenzwerte oder spezielle Betriebsmodi können so gezielt für einzelne Kanäle getestet werden.

Auch die Fehler- und Diagnoselogik ist pro Kanal gekapselt, sodass in der Simulation gezielt Fehlerbedingungen (z.B. DI_QI_BAD, DQ_QI_BAD) für einzelne Kanäle gesetzt und die Reaktion des Systems überprüft werden kann.

Durch diese modulare und strukturierte Architektur wird die Erstellung von Testbenches vereinfacht, die Testabdeckung erhöht und die Fehlersuche im Entwicklungsprozess deutlich beschleunigt.

Die zuvor beschriebene modulare Architektur in VHDL mit getrennten Steuer- und Datenpfaden lässt sich in der Simulation direkt überprüfen. Mithilfe der in VHDL implementierten Arrays und Schnittstellen (z.B. CTRL_IF, WR_REC) können im Testbench gezielt Stimuli gesetzt werden, um das Verhalten einzelner Kanäle zu validieren.

Die resultierenden Signalverläufe wurden in *Questa* aufgezeichnet. Abbildung ?? zeigt einen Ausschnitt dieser Wellenanalyse, in der sowohl die zyklischen Steuerbits als auch die azyklischen Schnittstellen WR_REC und RD_REC sichtbar sind.

Gut erkennbar ist, dass die zyklischen Steuerbits (z.B. CTRL_IF0, CTRL_IF1) in jedem SPS-Zyklus gesetzt und über das Prozessabbild verarbeitet werden. Parallel dazu laufen azyklische Operationen über die WR_REC- und RD_REC-Schnittstellen ab, die nur bei Bedarf aktiviert werden.

Im dargestellten Beispiel wird ein LoadRequest über die WR_REC-Schnittstelle angestoßen. Dieser führt dazu, dass die entsprechenden Steuerbits aktiv werden und anschließend im Zähler j_enc_count Änderungen auftreten. Auf diese Weise lässt sich in der Simulation nachvollziehen, wie deterministisch die zyklischen Steuerbefehle (z.B. Load, StartDQ) verarbeitet und wie azyklische Kommandos synchron eingesteuert werden.

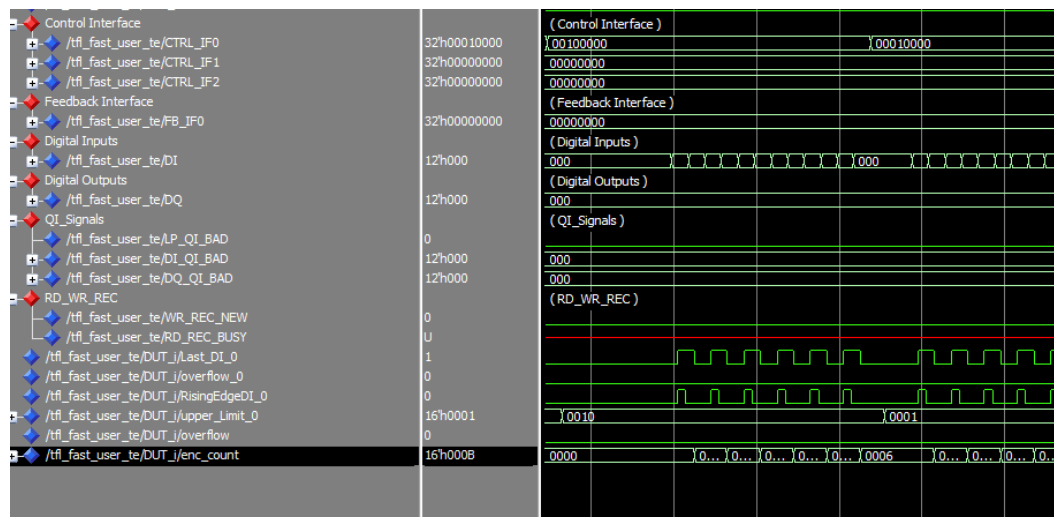


Abbildung 4.2: Wellenanalyse der CTRL_IF-, WR_REC- und RD_REC-Schnittstellen in Questa

4.12 Einbindung in das TIA Portal

Die Einbindung der entwickelten Applikationsbausteine in das TIA Portal ist ein wesentlicher Schritt, um die entwickelte Funktionalität praxisnah nutzbar zu machen. Dazu werden Funktionsbausteine erstellt, Schnittstellen mit dem TM FAST definiert und die Anbindung an die Prozessabbildung umgesetzt.

Ein zentrales Element ist der Baustein `LTMFAST_ControlREC`, der die azyklische Steuerung übernimmt. Er erhält alle notwendigen Informationen von den Funktionsbausteinen `UserReadREC` und `UserWriteREC`. Darüber werden beispielsweise Zustände wie „Kommunikation gestartet“, „Kommunikation läuft“, „Kommunikation erfolgreich abgeschlossen“, „Fehler aufgetreten“ sowie die zu übertragenden Nutzdaten abgebildet. Für jeden Funktionsbaustein muss eine Instanz angelegt werden, die im Projekt explizit mitgeführt wird (vgl. Listing ??).

Listing 4.8: Funktionsaufruf von #LTMFAST_UserWriteREC Instance

```

1 #LTMFAST_UserWriteREC_Instance(
2     req                := #loadrequest ,
3     busy               => #Busy ,
4     done               => #Done ,
5     error              => #Error ,
6     status             => #Status ,

```

```

7      userDataContent := #userDataContent ,
8      TMFASTControlREC :=
9      "TestHandler_LTMFAST_ControlREC_DB".LTMFAST_ControlREC_Instance
10 );

```

Für die zyklische Kommunikation ist der Baustein `LTMFAST_dosing_application` verantwortlich. Er bildet die Schnittstelle zum TM FAST-Modul und setzt die Eingangssignale in das vorgesehene Bitmuster um (vgl. Listing ??). Diese Signale werden über die I/O-Schnittstelle übertragen und mit den Steuerbits `Hold`, `Load`, `CounterReset`, `StartDQ`, `Select_Feedback_Value`, `polarity` und `EnabledQ` abgebildet.

Listing 4.9: Generierung der Steuerbits für die zyklische Kommunikation

```

1  FOR #ChannelIdx := 1 TO "Number_of_activ_Channels" DO
2
3      #AktChannel := #Channels[#ChannelIdx];
4
5      #Temp_CTRL_Bits[#ChannelIdx] := WORD#16#0000;
6      #LoadRequestArray[#ChannelIdx] := FALSE;
7
8      IF #AktChannel.Hold = TRUE THEN
9          #Temp_CTRL_Bits[#ChannelIdx] := WORD#16#1000;
10     ELSIF #AktChannel.CounterReset = TRUE THEN
11         #Temp_CTRL_Bits[#ChannelIdx] := WORD#16#0100;
12     ELSIF #AktChannel.Load = TRUE THEN
13         #Temp_CTRL_Bits[#ChannelIdx] := WORD#16#0800;
14         #LoadRequestArray[#ChannelIdx] := TRUE;
15     ELSIF #AktChannel.StartDQ = TRUE THEN
16         #Temp_CTRL_Bits[#ChannelIdx] := WORD#16#2000;
17     END_IF;
18
19     IF #AktChannel.EnabledQ = TRUE THEN
20         #Temp_CTRL_Bits[#ChannelIdx] :=
21         #Temp_CTRL_Bits[#ChannelIdx] OR WORD#16#0400;
22     END_IF;
23
24     IF #AktChannel.polarity = TRUE THEN

```

```

25     #Temp_CTRL_Bits[#ChannelIdx] :=
26     #Temp_CTRL_Bits[#ChannelIdx] OR WORD#16#0200;
27 END_IF;
28
29 IF #AktChannel.Select_Feedback_Value = TRUE THEN
30     #Temp_CTRL_Bits[#ChannelIdx] :=
31     #Temp_CTRL_Bits[#ChannelIdx] OR WORD#16#4000;
32 END_IF;
33
34 END_FOR;

```

Wie in Abbildung ?? dargestellt, wird der Funktionsbaustein direkt im OB1 eingebunden und über einen selbst definierten Datentyp (UDT) **Channels** angesteuert. Dieser Datentyp enthält alle für die zyklische Kommunikation relevanten Steuerungen.

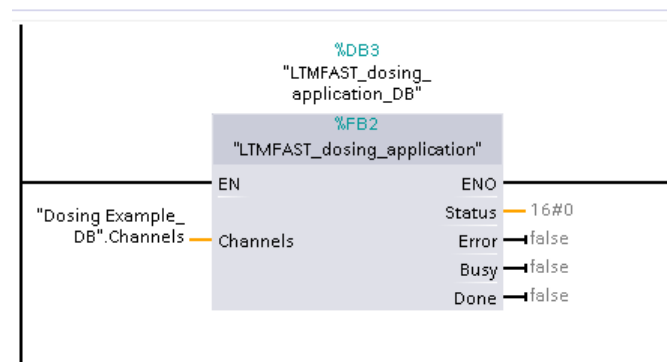


Abbildung 4.3: Integration des Applikationsbausteins in OB1

Eine typische Steuerung des **LTMFAST_dosing_application**-Bausteins besteht darin, azyklische Daten in den zugehörigen Instanzdatenbaustein einzutragen und gleichzeitig die zyklischen Steuerungen der gewünschten Kanäle zu setzen. Ein Anwendungsbeispiel ist in Listing ?? gezeigt.

Listing 4.10: Beispiel zur Steuerung der zyklischen und azyklischen Kommunikation

```

1 IF #LoadFlag = 1 THEN
2     #Channels[1].Load := 0;
3 END_IF;

```

```

4
5 IF #StartDQFlag = 1 THEN
6     #Channels[1].StartDQ := 0;
7 END_IF;
8
9 IF #Init = 0 THEN
10     #Init := 1;
11     #Channels[1].EnableDQ := 1;
12     "LTMFAST_dosing_application_DB".userDataContent[1] :=
13     16#3E8;
14     "LTMFAST_dosing_application_DB".userDataContent[15] :=
15     16#1;
16     #Channels[1].Load := 1;
17     #LoadFlag := 1;
18 END_IF;
19
20 IF ("FAST_FB".FB_IF_Channel[1] AND 16#8000) <> 0 THEN
21     #Channels[1].StartDQ := 1;
22     #StartDQFlag := 1;
23 END_IF;

```

4.13 Vergleich mit bisherigen Lösungen

ist das Teil der Entwicklung oder eher Fazit und Ausblick?

Im Vergleich zur FM350-2 bietet das neue Design eine höhere Flexibilität, bessere Ressourcenausnutzung und einfachere Erweiterbarkeit. Die klare Trennung von Steuer-, Daten- und Diagnosepfaden sowie die flexible Kanaluordnung ermöglichen eine optimale Anpassung an unterschiedliche Applikationen.

(Implementierung des Frequenzmessers / Vergleich von FM350-2 (ist im Handbuch gut erklärt) und TMFAST)

5 Fazit und Ausblick

5.1 Zusammenfassung der Ergebnisse

Im Rahmen dieser Arbeit konnte gezeigt werden, dass alle für Dosieranlagen relevanten Funktionen erfolgreich in das neue VHDL-Design integriert wurden. Ein wesentliches Ergebnis ist die drastische Verkürzung der Reaktionszeit: Während die bisher eingesetzte Baugruppe FM 350-2 eine Verzögerung von rund 50 μ s aufwies, erreicht die neue Implementierung eine Reaktionszeit von 20 ns. Damit wird eine sofortige und deterministische Signalverarbeitung ermöglicht, was insbesondere in zeitkritischen Dosierprozessen erhebliche Vorteile bringt.

Ein Unterschied zur alten Baugruppe liegt in der Behandlung der Prozessalarme. Viele der in der FM 350-2 implementierten Alarmer konnten in der neuen Architektur nicht in gleicher Form umgesetzt werden. Während die FM 350-2 interne, externe und kanalspezifische Fehler durch Prozessalarmer meldete und dabei auch eine Sammelfehler-LED (SF) sowie Diagnose-OBs in der SPS nutzte, verfolgt das neue VHDL-Design einen anderen Ansatz.

Die Fehler- und Diagnosestrategie wurde systematisch auf Kanalebene implementiert. Für jeden Kanal wird ein Signal `system_fault(i)` berechnet, das Fehler an Eingängen, Ausgängen, RS485-Schnittstellen oder der Spannungsversorgung überwacht. Erkennt die Logik einen Fehler, wird der betroffene Ausgang unmittelbar in den sicheren Zustand (SAFE State) überführt. Die Details werden über das Feedback-Interface an die SPS zurückgemeldet, sodass dort eine gezielte Reaktion erfolgen kann. Im Fall eines globalen CPU-Stop werden zusätzlich alle Kanäle zurückgesetzt und sämtliche Ausgänge deaktiviert. Auf diese Weise wird sichergestellt, dass sowohl Einzelkanalfehler als auch globale Störungen zuverlässig erkannt und behandelt werden.

5.2 Kritische Reflexion und Herausforderungen

Ein zentrales Problem bestand darin, dass kein realer Aufbau der alten FM 350-2-Baugruppe verfügbar war. Vergleichsdaten mussten daher ausschließlich aus der Dokumentation und dem Handbuch entnommen werden. Dadurch konnten bestimmte Verhalten nur indirekt nachvollzogen werden, während praktische Messungen oder Tests zum Vergleich der Reaktionszeiten und Fehlerdiagnosen nicht möglich waren. Diese Einschränkung führte dazu, dass die Analyse in Teilen theoretisch blieb und auf Annahmen basierte, die nicht vollständig verifiziert werden konnten.

5.3 Potenzielle Weiterentwicklungen und zukünftige Anwendungen

Ein wichtiger Ansatzpunkt für zukünftige Arbeiten ist die Integration von Prozessalarmen, wie sie in der FM 350-2 vorgesehen waren. Diese könnten über eine geeignete Erweiterung des azyklischen Datensatzes `RD_REC` realisiert werden, um sowohl detaillierte Fehlerinformationen als auch Diagnosehinweise sicher zu übertragen. Ziel sollte es sein, die Vorteile der neuen Architektur – insbesondere die extrem kurze Reaktionszeit und die sichere Fehlerbehandlung – mit der umfangreichen Diagnosetiefe der alten Baugruppe zu kombinieren.

Darüber hinaus bietet das flexible VHDL-Design eine solide Grundlage für weitere Applikationen. Durch die Anpassung der Kanalzuordnung und die Erweiterung der Kommunikationsschnittstellen können neue Einsatzfelder erschlossen werden, beispielsweise für hochpräzise Dosier- oder Zählssysteme in unterschiedlichen industriellen Bereichen. Auch die Anbindung an moderne Steuerungssysteme über standardisierte Protokolle stellt ein mögliches Entwicklungsfeld dar, um die Integration in komplexe Automatisierungsumgebungen weiter zu vereinfachen.

Abbildungsverzeichnis

Tabellenverzeichnis

Literaturverzeichnis

- [1] Liam Bee (2022): *PLC and HMI development with Siemens TIA Portal: develop PLC and HMI programs using standard methods and structured approaches with TIA Portal V17*. Packt Publishing, Limited.
- [2] Siemens AG (2011): *Zählerbaugruppe FM 350-2 Gerätehandbuch*. Verfügbar unter: https://cache.industry.siemens.com/dl/files/178/1105178/att_20226/v1/s7300_fm350_2_operating_instructions_de_de-DE.pdf. Zugriff: 11. März 2025.
- [3] Siemens AG (2023): *TMFast Programmeier Handbuch*. Verfügbar unter: https://cache.industry.siemens.com/dl/files/088/109816088/att_1144842/v2/s71500_tm_fast_programming_manual_de-DE_de-DE.pdf. Zugriff: 12. März 2025.
- [4] Wolfgang Bable: *Die Automatisierungspyramide von 1985 – Grundlegende Struktur in IoT und Industrie 4.0*
- [5] H. Berger: *Automatisieren mit SIMATIC S7-1500: Projektieren, Programmieren und Testen mit STEP 7 Professional*. 2. Aufl. Erlangen: Publics Publishing, 2017.
- [6] W. Babel: *Systemintegration in Industrie 4.0 und IoT: Vom Ethernet bis hin zum Internet und OPC UA*. Wiesbaden: Springer Fachmedien, 2024. DOI: <https://doi.org/10.1007/978-3-658-42987-4>.
- [7] Siemens AG: *Totally Integrated Automation Portal*. siemens.com. Verfügbar unter: <https://new.siemens.com/de/de/produkte/automatisierung/industrie-software/automatisierungs-software/tia-portal.html> (abgerufen am 26.07.2025).
- [8] Siemens AG: *SIMATIC Energy Suite*. siemens.com. Verfügbar unter: <https://www.siemens.com/de/de/produkte/automatisierung/>

- [industrie-software/automatisierungs-software/energiemanagement/simatic-energy-suite.html](#) (abgerufen am 26.07.2025).
- [9] Siemens AG: *SIMATIC ET 200MP*. siemens.com. Verfügbar unter: <https://new.siemens.com/de/de/produkte/automatisierung/systeme/industrie/io-systeme/simatic-et-200mp.html> (abgerufen am 26.07.2025).
- [10] Intel® Corp.: *Intel Cyclone 10 LP Device Overview*, 2022. [Online] Verfügbar unter: <https://www.intel.com/content/www/us/en/docs/programmable/683879/current/device-overview.html> (abgerufen am 27.07.2025).
- [11] G. Wild *et al.*: *Feature Specification for TM FAST*, Version 2.0, 2025.
- [12] Encoder Products Company: *The Basics of How an Encoder Works*, White Paper (WP-2011), rev. 15/08/25, 2025. [Online] Verfügbar unter: https://www.encoder.com/hubfs/white-papers/WP-2011_Basics/wp2011-basics-how-an-encoder-works.pdf (abgerufen am 15. August 2025).
- [13] Sensoray, Inc.: *Introduction to Incremental Encoders*, Application Note. [Online] Verfügbar unter: <https://www.sensoray.com/support/appnotes/encoders.htm> (abgerufen am 15. August 2025).
- [14] JUMO GmbH & Co. KG: *Regelungstechnik – Grundlagen und Anwendungen*. Verfügbar unter: https://www.spsHaus.ch/files/inc/Downloads/Lernumgebung/Downloads/Allgemein/Regelungstechnik_Jumo.pdf (abgerufen am 03. September 2025).
- [15] Prof. Dr.-Ing. K. Stephan: *Automatisierung in der Prozesstechnik – Lehrstoffsammlung, Kapitel I*. Verfügbar unter: <https://automatisierung-stephan.de/wp-content/uploads/2024/07/Automatisierung-in-der-Prozesstechnik-I.pdf> (abgerufen am 05. September 2025).
- [16] Hering, E.; Endres, J.; Gutekunst, J. (Hrsg.): *Elektronik für Ingenieure und Naturwissenschaftler*. Springer Vieweg, Wiesbaden.
- [17] Siemens AG: *Function Blocks, OBs, FCs, and DBs in SIMATIC S7-1200 — Programming Concepts*. Online-Dokumentation. Verfügbar unter: https://docs.tia.siemens.cloud/r/simatic_s7_1200_manual_collection_enus_

- [20/programming-concepts/using-blocks-to-structure-your-program/function-block-fb](#) (abgerufen am 05. September 2025).
- [18] Heinrich, B.; Linke, P.; Glöckler, M.: *Grundlagen Automatisierung*. Hanser Verlag, München, 2018.
- [19] Siemens AG: *High Speed Boolean Processor FM 352-5*. Verfügbar unter: <https://mall.industry.siemens.com/mall/de/b1/Catalog/Products/10000781?activeTab=product> (abgerufen am 06. September 2025).
- [20] Intel® Corporation: *Intel Quartus Prime Standard Edition Handbook Volume 1: Design and Compilation*, 2023. [Online] Verfügbar unter: <https://www.intel.com/content/www/us/en/docs/programmable/683082/current/overview.html> (abgerufen am 27.07.2025).
- [21] Intel® Corporation: *Intel Quartus Prime Software*. [Online] Verfügbar unter: <https://www.intel.de/content/www/de/de/products/details/fpga/development-tools/quartus-prime.html> (abgerufen am 27.07.2025).
- [22] Intel® Corporation: *Questa*-Intel® FPGA Edition Software*. [Online] Verfügbar unter: <https://www.intel.de/content/www/de/de/software/programmable/quartus-prime/questa-edition.html> (abgerufen am 27.08.2025).
- [23] Siemens AG: *SIMATIC S7-1500 Structure and Use of the CPU Memory Function Manual*. Version 01/2013, Dokument-Nr. A5E03461664-01. [PDF] Verfügbar unter: https://cache.industry.siemens.com/dl/files/101/59193101/att_80162/v1/s71500_structure_and_use_of_the_PLC_memory_function_manual_en-US_en-US.pdf (abgerufen am 9.09.2025).
- [24] Siemens AG: *SIMATIC Safety Integrated – machine safety seamlessly integrated*. Webseite. Verfügbar unter: <https://www.siemens.com/global/en/products/automation/topic-areas/safety-integrated/factory-automation/offering/simatic-safety.html> (abgerufen am 06. September 2025).